

Computer Vision in Python

Day 2, Part 1/4

Matthieu De Mari



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN

About this lecture

1. What is **interpretability** in Deep Learning?
2. What are **attacks** on Neural Networks (NNs) and what is **Adversarial Machine Learning**?
3. Why are attacks an **important concept** when studying NNs and what are the important lessons to be learnt from Adversarial ML?
4. What are the different **types of attacks** and what is the intuition behind basic attacks?
5. How to **defend** against such attacks?
6. **State-of-the-art** of attacks and defense, **open questions** in research, ethical discussions.

What is interpretability? (or eq. Explainability)

Definition (Interpretability** in Neural Networks):**

Interpretability is the degree to which a human can understand the cause of a decision, i.e. the degree to which a human can consistently predict the model's result.

The higher the interpretability of a machine learning model, the easier it is for someone to comprehend why certain decisions or predictions have been made.

Typically, a model is more interpretable than another model if its decisions are easier for a human to comprehend than decisions from the other model.

Why do we need interpretability?

There are many reasons why we need interpretability.

- **Reason #1:** Because humans are easily scared by things they do not understand and do not like things that are not easily explainable.
- **Reason #2:** Improving Neural Networks decisions and training methods.
- **Reason #3:** Confirming what a Neural Network has learnt and what it seems to implement to reach a decision.
- **Reason #4:** Identify the reasons for mistakes and cognitive biases in the decisions of a Neural Network in an attempt to fix them.

Reason #1: Because humans do not like things that are not easily explainable.

Should We Be Afraid of AI?



Ron Schmelzer Contributor

COGNITIVE WORLD Contributor Group ⓘ

AI

Philosophers, computer scientists, and even Elon Musk are concerned artificial intelligence could one day destroy the world. Some scholars argue it's is the most pressing existential risk humanity might ever face, while others mostly dismiss the hypothesized danger as unfounded doom-mongering.

<https://theconversation.com/people-dont-trust-ai-heres-how-we-can-change-that-87129>

<https://www.vice.com/en/article/7x48kg/the-divide-between-people-who-hate-and-love-artificial-intelligence-is-not-real>

Reason #1: Because humans do not like things that are not easily explainable.



Reason #1
that are n

Should We



Ron Schmelzer
COGNITIVE WORK
AI

Philosophy
artificial
it's is t
others
monge

Letter signed by Elon Musk demanding AI research pause sparks controversy

The statement has been revealed to have false signatures and researchers have condemned its use of their work



A letter called for a six-month pause on development of systems more powerful than GPT-4, developed by OpenAI, a company co-founded by Elon Musk. Photograph: Michael Dwyer/AP

A letter co-signed by Elon Musk and thousands of others demanding a pause in artificial intelligence research has created a firestorm, after the researchers

like things

are concerned
Some scholars argue
ever face, while
unded doom-

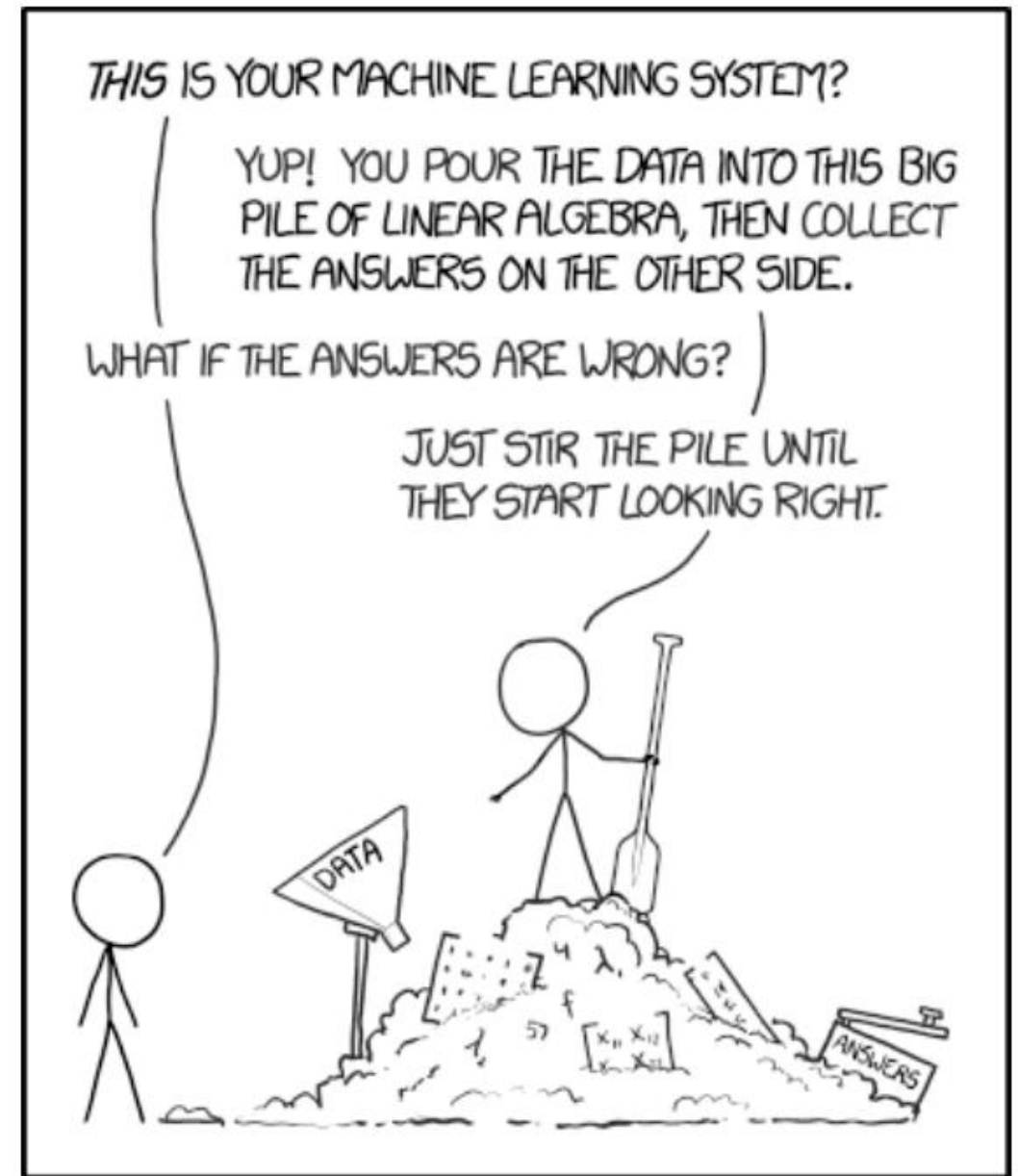
<https://theconversation.com/article/7x48kg/the-who-hate-and-love-not-real>

<https://theconversation.com/article/7x48kg/the-who-hate-and-love-not-real>
trust-ai-heres-how-we

Reason #2

Reason #2: Improving Neural Networks decisions and training methods.

- Neural Networks are black-boxes, and a very empirical science.
- Experience shows that these algorithms have been able to (partially) answer some problems we had no clue how to address.
- But still very obscure...

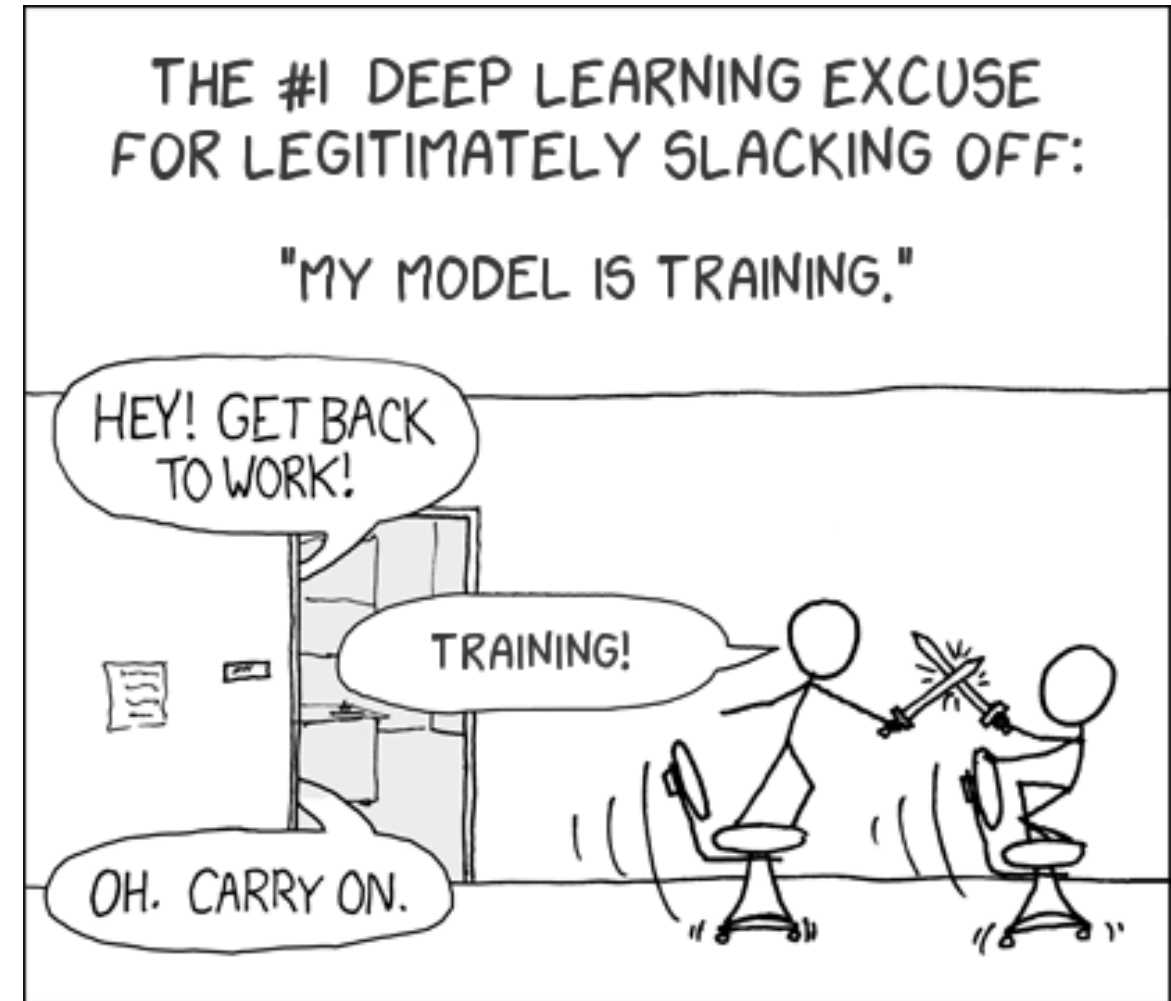


[https://www.explainxkcd.com/wiki/index.php/1838: Machine Learning](https://www.explainxkcd.com/wiki/index.php/1838: Machine_Learning)

Reason #2

Reason #2: Improving Neural Networks decisions and training methods.

- Understanding how NNs tend to learn could lead to more efficient training procedures.
- (Right now we kind of pray that an empirically validated model architecture and some Gradient Descent will make it work?)

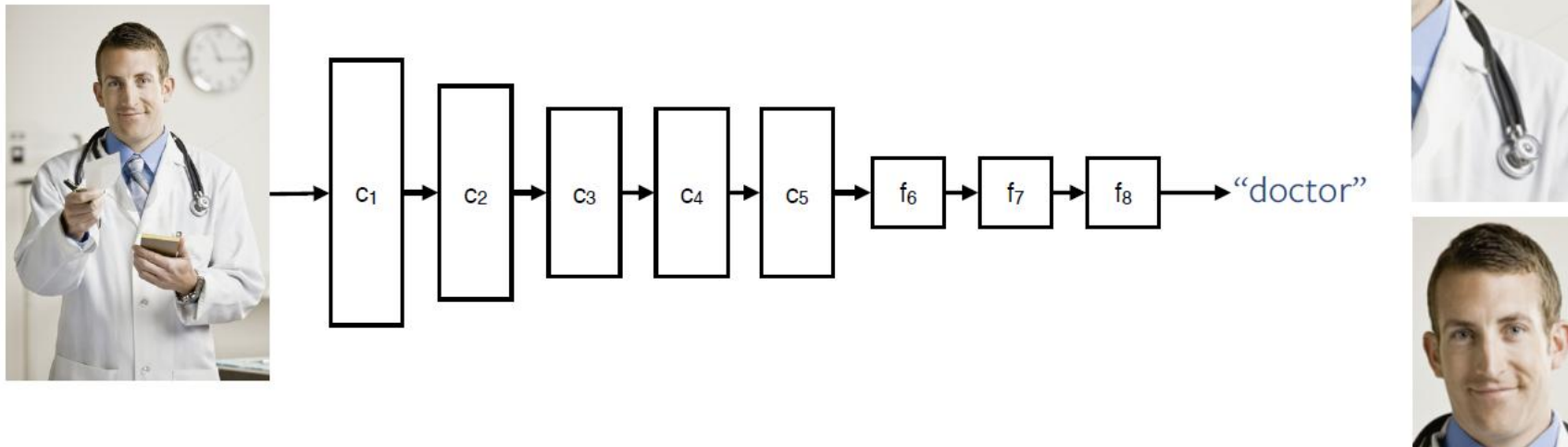


[https://www.explainxkcd.com/wiki/index.php/303: Compiling](https://www.explainxkcd.com/wiki/index.php/303:_Compiling)

Reason #3

Reason #3: Confirming what a Neural Network has learnt and what it seems to implement to reach a decision.

- What features is the neural network learning to reach a decision?
- What similarities is the neural network looking for in the dataset?
- What are the neurons doing anyway?



Reason #4

Reason #4: Identify the reasons for mistakes and cognitive biases in the decisions of a Neural Network in an attempt to fix them.

Definition (cognitive bias):

Similar to cognitive biases in psychology.

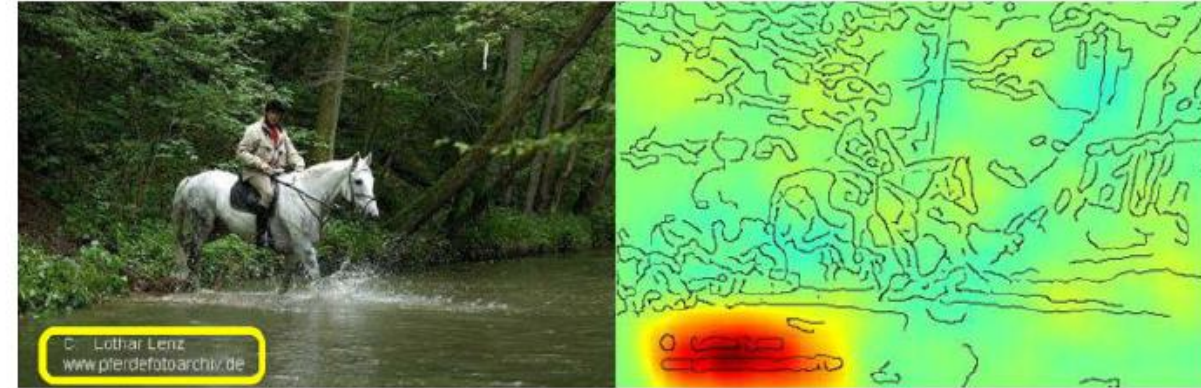
In Neural Networks, a **cognitive bias** refers to a feature, which has been used by the NN to reach a certain decision, but should not have been used as it seems illogical to humans.

Reason #4

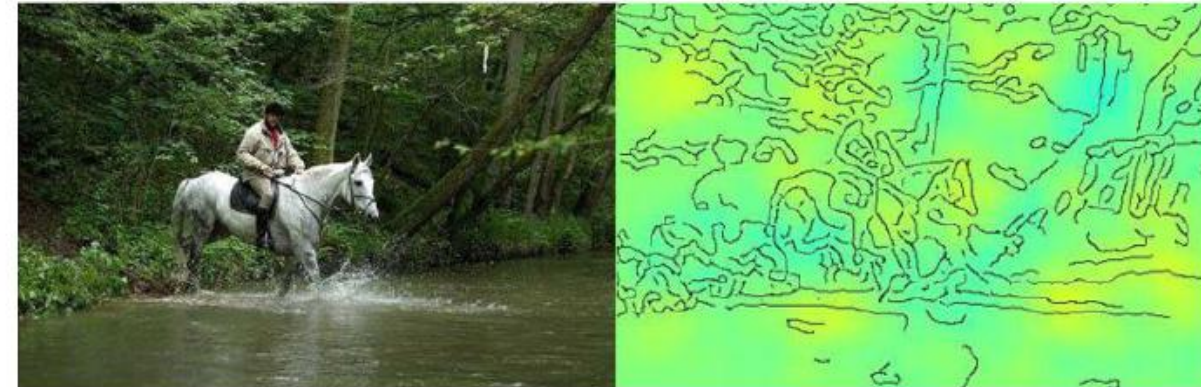
Reason #4: Identify the reasons for mistakes and cognitive biases in the decisions of a Neural Network in an attempt to fix them.

- The Neural network on the right draws information from the photo legend and recognizes that this photographer likes to take pictures of horses...
- That is a cognitive bias!

“horse”

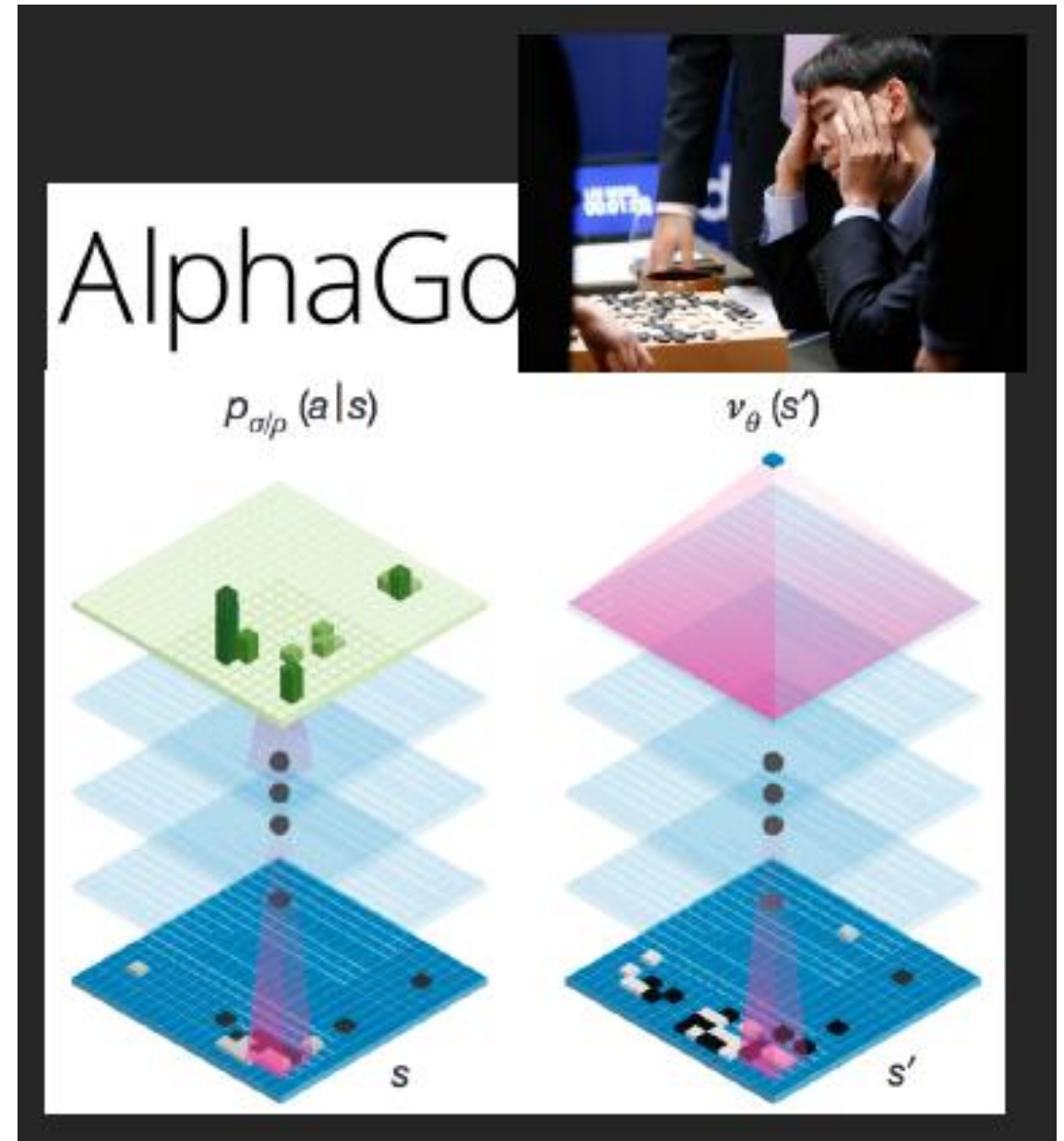


“not horse”



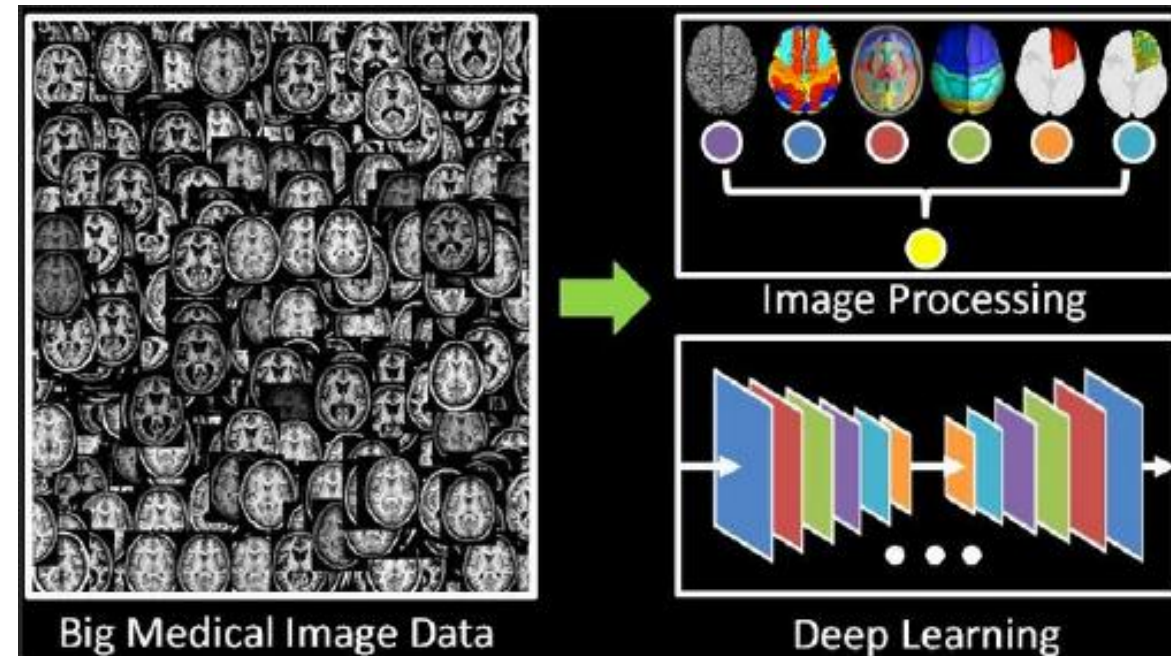
Reason #4

- **Causality:** Check that only causal relationships are picked up. No cognitive biases.



Reason #4

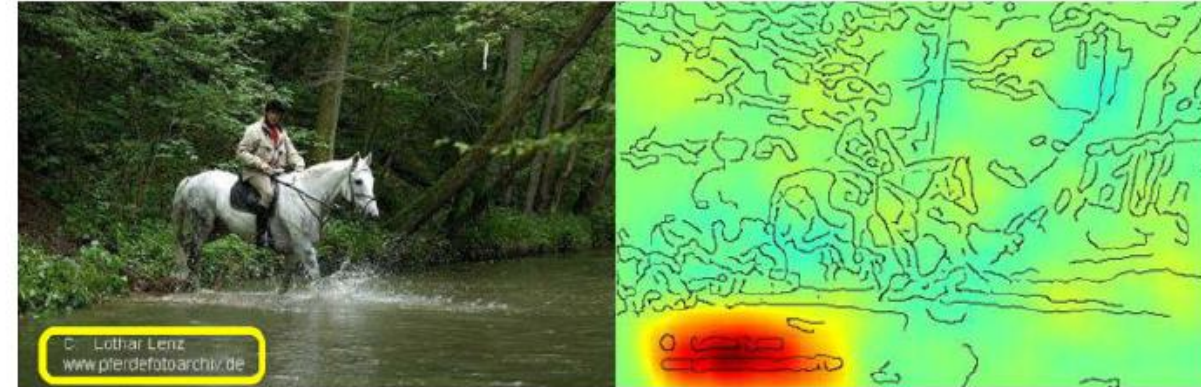
- **Causality:** Check that only causal relationships are picked up. No cognitive biases.



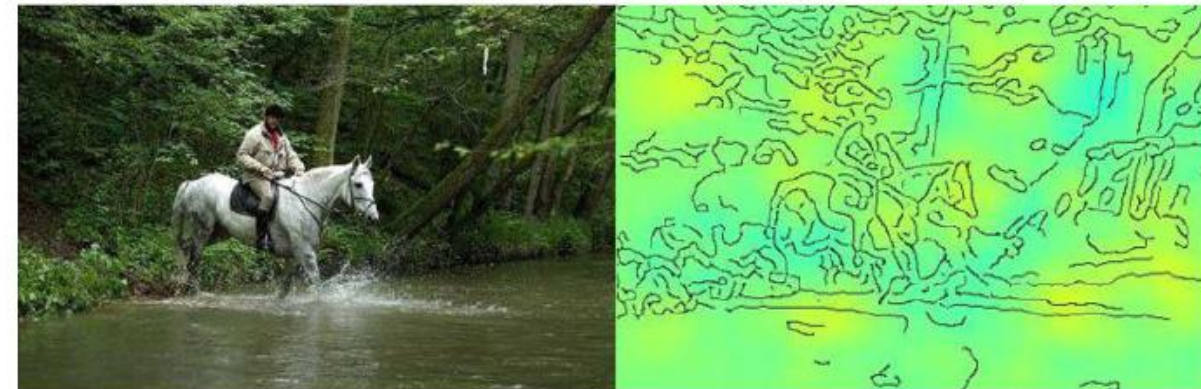
Reason #4

- **Causality:** Check that only causal relationships are picked up. No cognitive biases.
- **Reliability or Robustness:** Ensuring that small changes in the input do not lead to large changes in the prediction.

“horse”

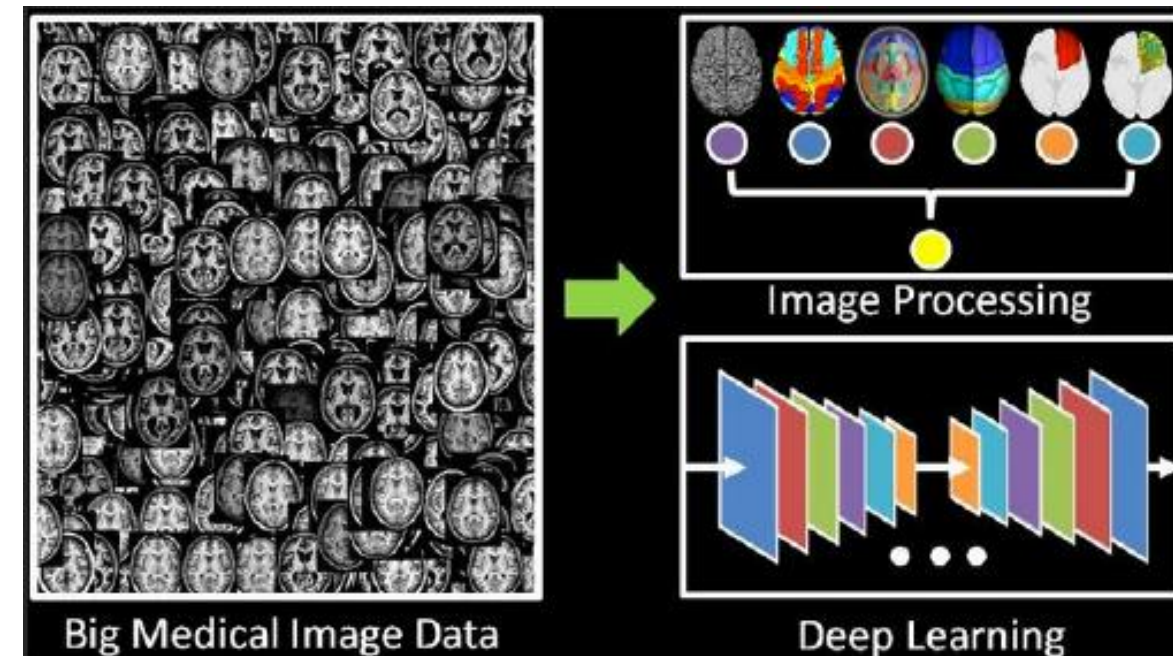


“not horse”



Reason #4

- **Causality:** Check that only causal relationships are picked up. No cognitive biases.
- **Reliability or Robustness:** Ensuring that small changes in the input do not lead to large changes in the prediction.



Reason #4

- **Causality:** Check that only causal relationships are picked up. No cognitive biases.
- **Reliability or Robustness:** Ensuring that small changes in the input do not lead to large changes in the prediction.
- **Privacy:** Ensuring that sensitive information in the data is protected.



Reason #4

- **Fairness:** Ensuring that predictions are unbiased and do not implicitly or explicitly discriminate against underrepresented groups.
- An interpretable model can tell you why it has decided that a certain person should not get a loan, and it becomes easier for a human to judge whether the decision is based on a learned demographic (e.g. racial) bias.

NEWS • 24 OCTOBER 2019 • UPDATE 26 OCTOBER 2019

Millions of black people affected by racial bias in health-care algorithms

Study reveals rampant racism in decision-making software used by US hospitals – and highlights ways to correct it.



<https://www.nature.com/articles/d41586-019-03228-6>

Attacks: definition

Definition (Attacks on Neural Networks):

Adversarial machine learning, or attacks on Neural Networks, refers to machine learning techniques that attempt to fool trained models by supplying deceptive input.

The most common reason is to cause a malfunction in a machine learning model.

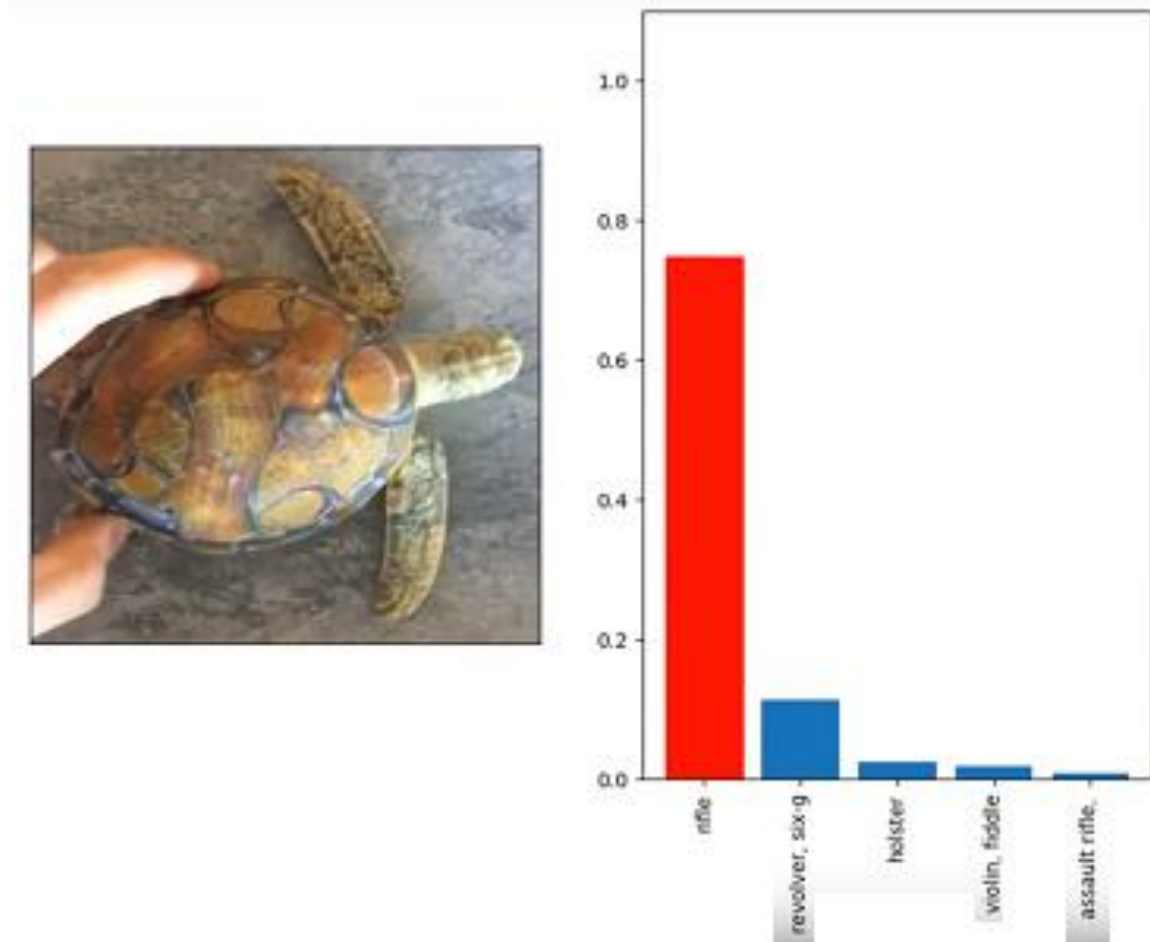


Attack/Adversarial samples: definition

Definition (**Attack samples, adversarial samples**):

An **input sample** is considered an **attack sample** (or **adversarial sample**) for a given trained model, if and only if, it makes this model malfunction on purpose.

Example: this picture of a turtle has been altered on some of its pixels to be misclassified as a weapon (rifle, revolver, etc.).



Source: Google's AI thinks this turtle looks like a gun, which is a problem [Verge1].

On the ethics of **attacking** Neural Networks

This week's lectures and notebooks will introduce techniques, whose objective is to **make a trained Neural Network malfunction on purpose**.

- These techniques are NOT, so to speak, illegal.
- But let us keep in mind what the consequences of using these attacks could be...

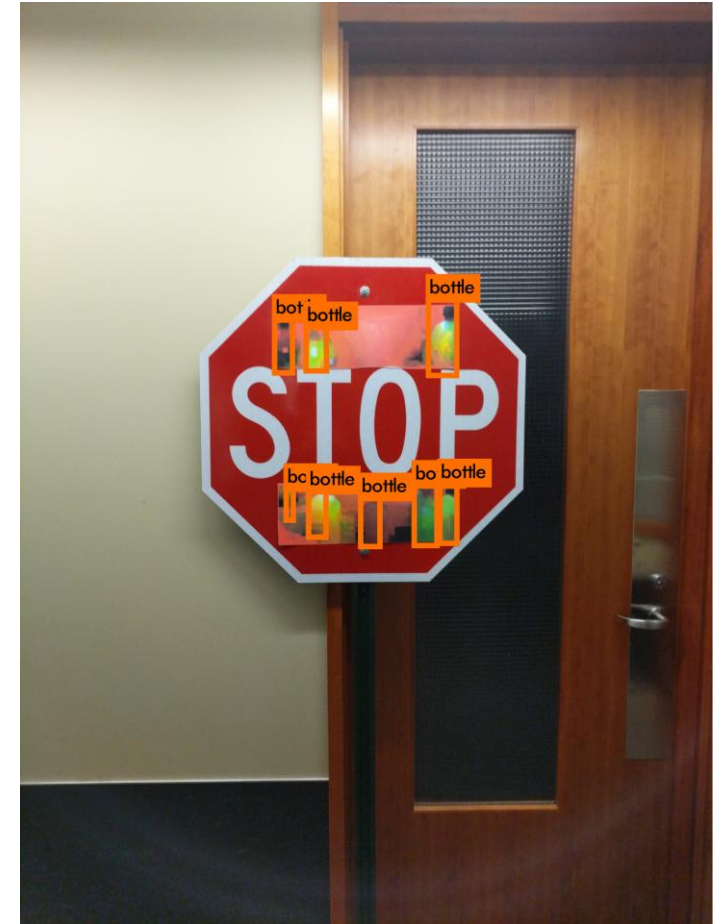


On the ethics of **attacking** Neural Networks

Example #1: This stop sign has stickers put in specific locations.

- It is an **attack sample** as it can no longer be detected as a stop sign, and is instead misclassified as several bottles.

Think: What would be the effect/consequence of such an attack sample on a self-driving car using computer vision?



Source: Slight Street Sign Modifications Can Completely Fool Machine Learning Algorithms [Spectrum1].

On the ethics of **attacking** Neural Networks

Example #2: Covering some areas of your face with paint or glasses with specific patterns can fool facial recognition algorithms.

- These facial recognition AIs are no longer able to detect a face, let alone recognize the identity of the person.

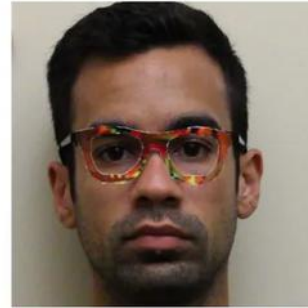
Think: Is that a good or a bad discovery for computer vision?



(b)



(c)

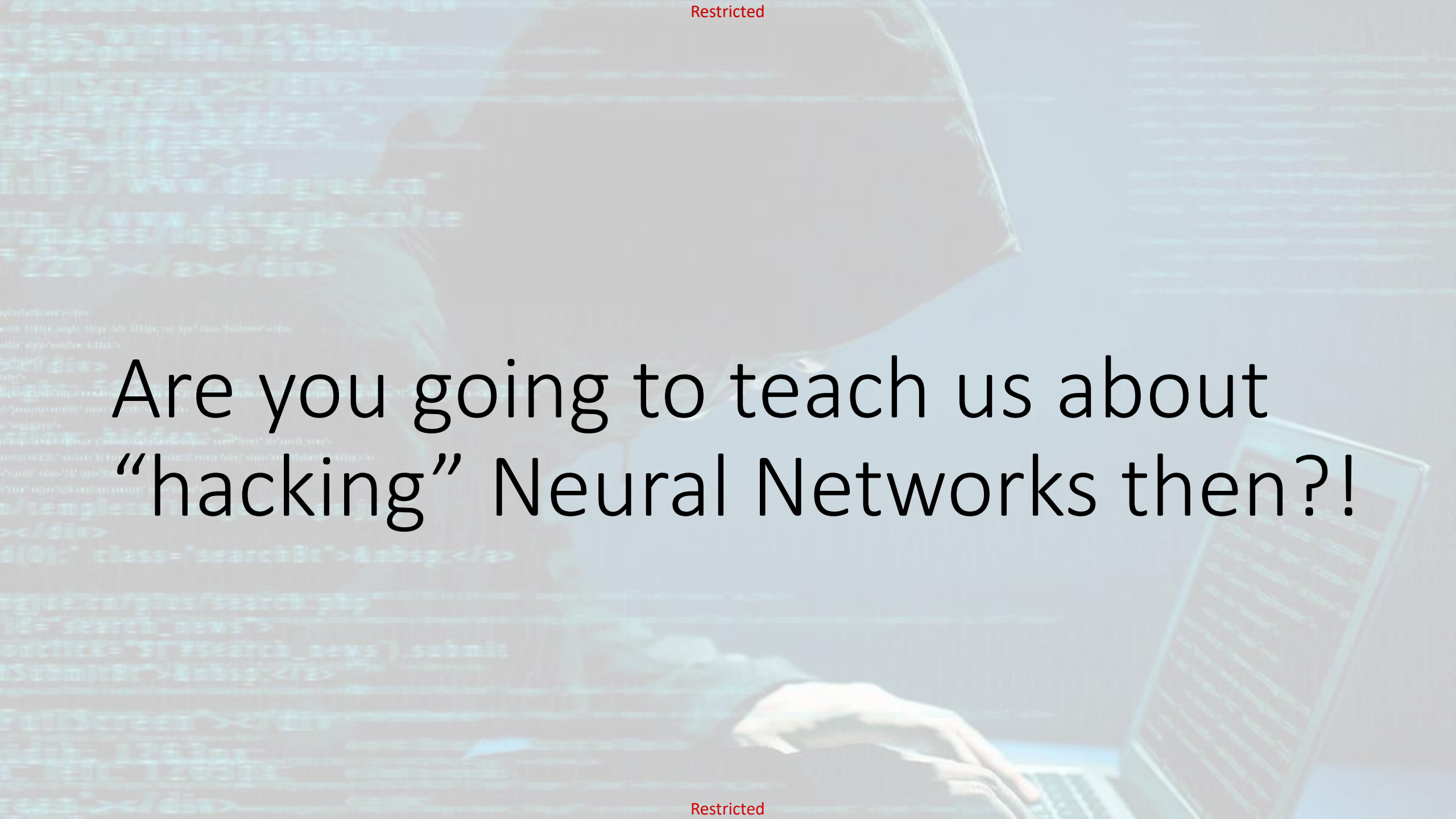


(d)

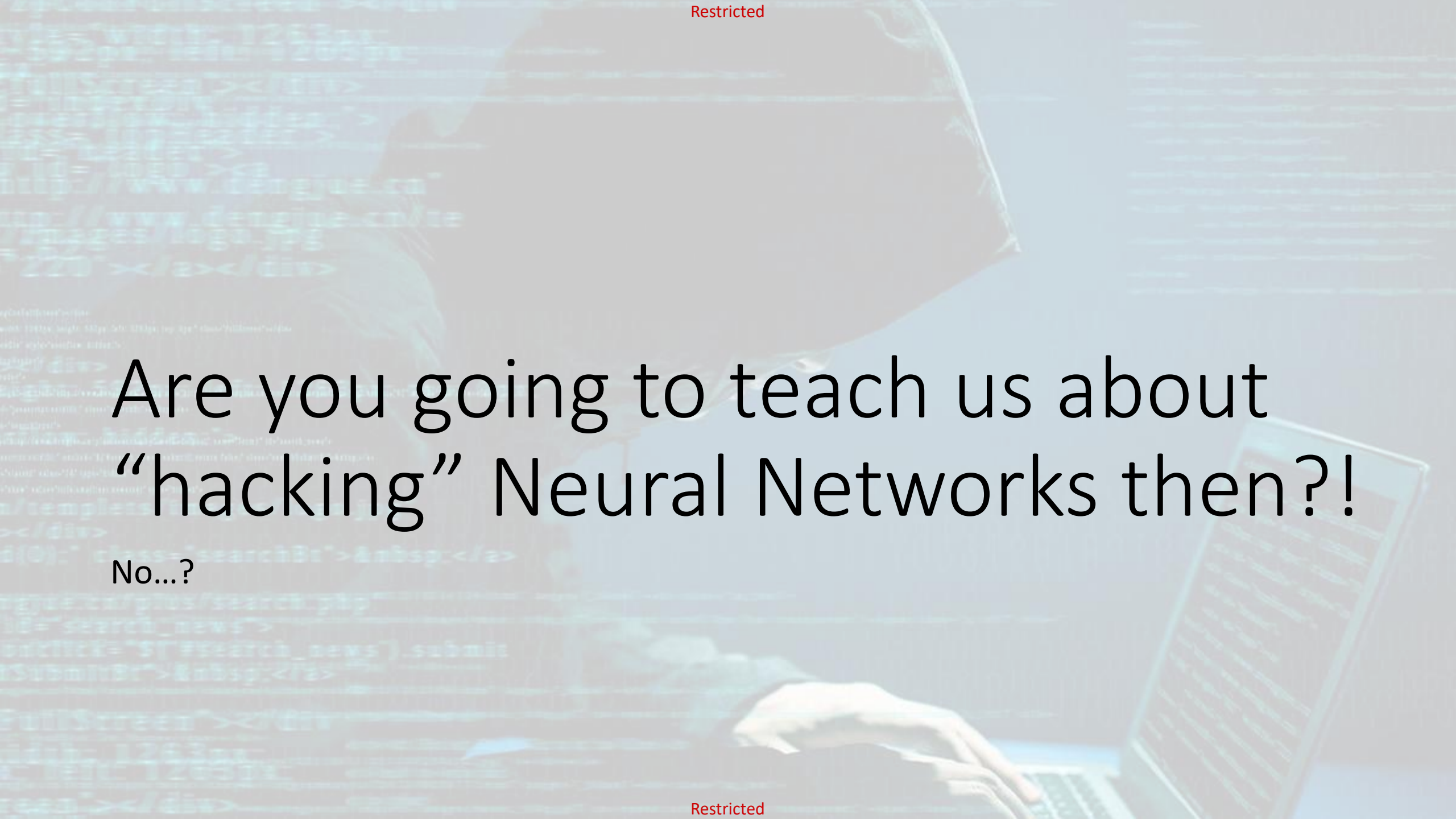


Source: These glasses trick facial recognition software into thinking you're someone else [Verge2].

Source: Defeating Facial Recognition [YTB1].

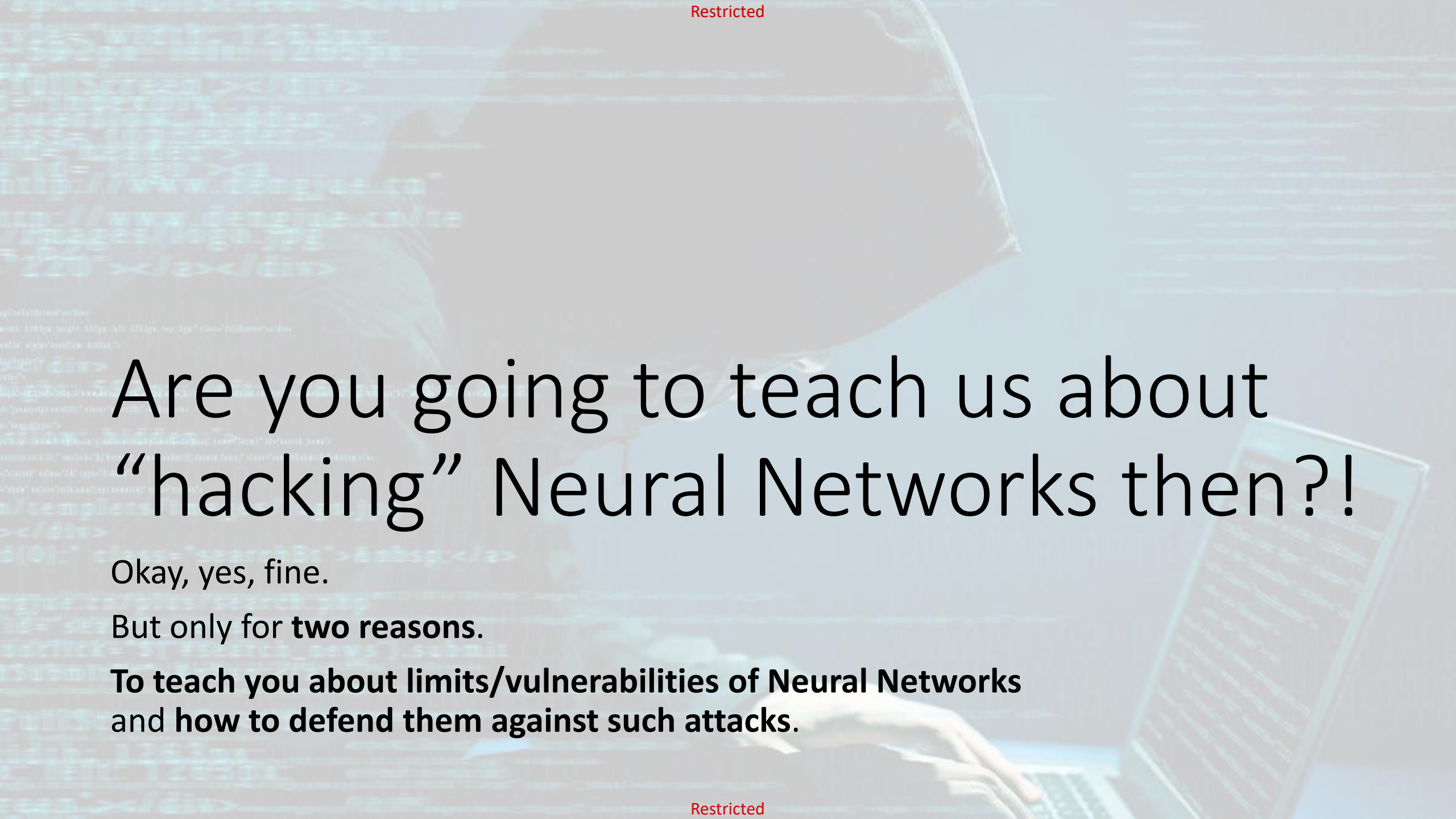
A person wearing a dark hoodie is shown from the side, typing on a laptop. The background is a light blue gradient with faint, semi-transparent elements: a world map, a laptop screen displaying code, and various lines of HTML and CSS code scattered across the scene. The overall aesthetic is tech-oriented and digital.

Are you going to teach us about
“hacking” Neural Networks then?!



Are you going to teach us about “hacking” Neural Networks then?!

No...?



Are you going to teach us about “hacking” Neural Networks then?!

Okay, yes, fine.

But only for **two** reasons.

**To teach you about limits/vulnerabilities of Neural Networks
and how to defend them against such attacks.**

Reason #1: Limits

Reason #1: Neural networks are **limited and vulnerable**, by design.

- They will always be at risk of attacks making them malfunction, no matter how many safeguards you decide to put in place.

Reason #1: Limits

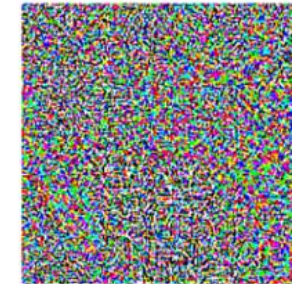
Reason #1: Neural networks are **limited and vulnerable**, by design.

- They will always be at risk of attacks making them malfunction, no matter how many safeguards you decide to put in place.
- For instance, adding noise to an image is often enough to fool any image recognition algorithm.



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=

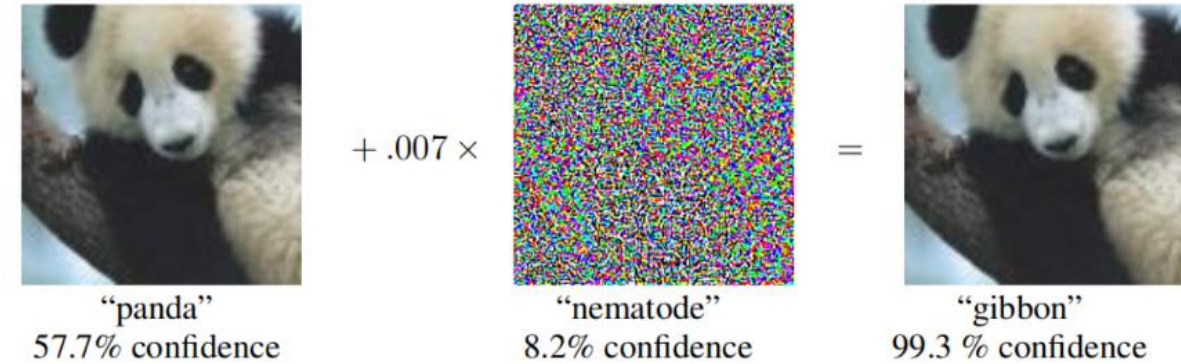


“gibbon”
99.3 % confidence

Reason #1: Limits

Reason #1: Neural networks are **limited and vulnerable**, by design.

- They will always be at risk of attacks making them malfunction, no matter how many safeguards you decide to put in place.
- For instance, adding noise to an image is often enough to fool any image recognition algorithm.

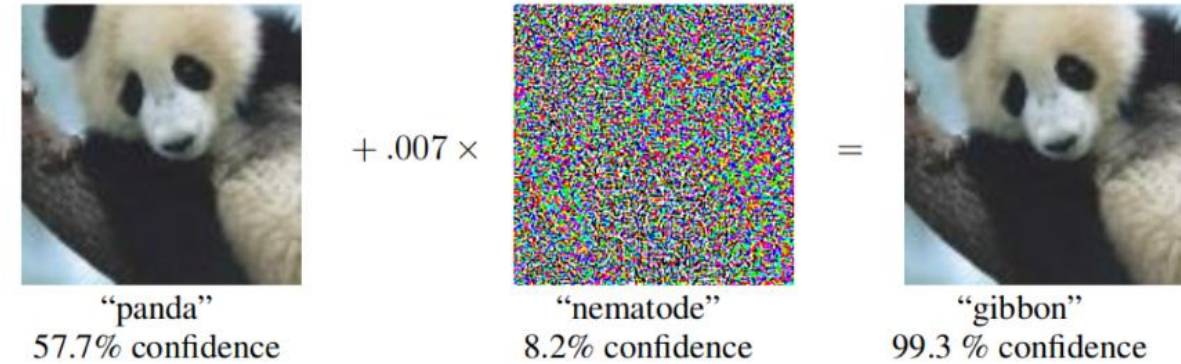


WHO WOULD WIN?

Reason #1: Limits

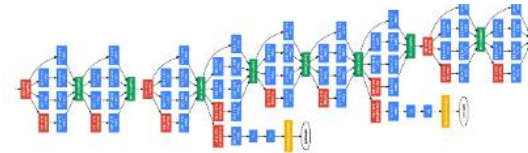
Reason #1: Neural networks are **limited and vulnerable**, by design.

- They will always be at risk of attacks making them malfunction, no matter how many safeguards you decide to put in place.
- For instance, adding noise to an image is often enough to fool any image recognition algorithm.



WHO WOULD WIN?

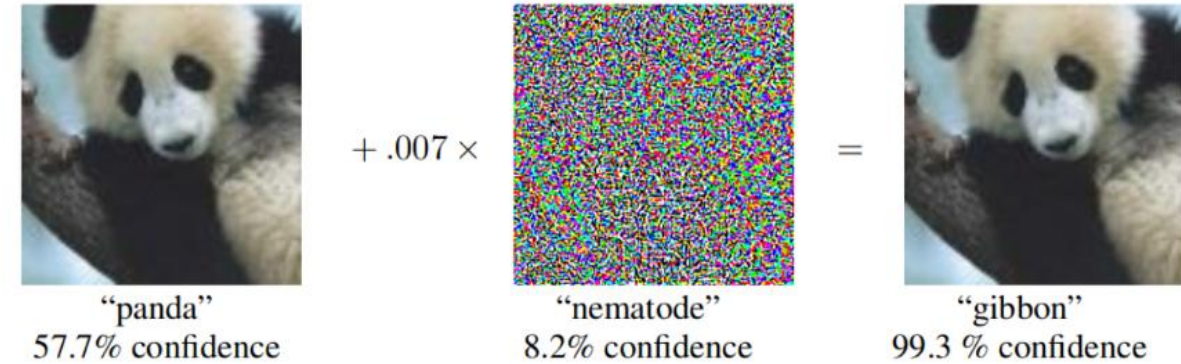
A deep convolutional network with 1 million parameters, trained for days on 64 GPUs, using a dataset consisting of millions of images



Reason #1: Limits

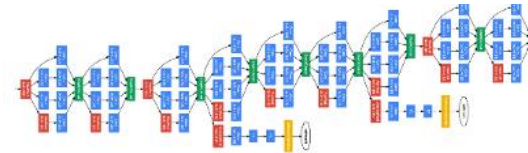
Reason #1: Neural networks are **limited and vulnerable**, by design.

- They will always be at risk of attacks making them malfunction, no matter how many safeguards you decide to put in place.
- For instance, adding noise to an image is often enough to fool any image recognition algorithm.



WHO WOULD WIN?

A deep convolutional network with 1 million parameters, trained for days on 64 GPUs, using a dataset consisting of millions of images



A tiny bit of noise, added to an original image



Reason #1: Limits

Reason #1: Neural networks are limited



$+ .007 \times$



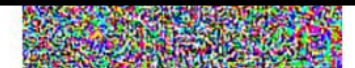
$=$



- That at minimum, the purpose of the model is to learn the underlying structure of the data.
- For image classification, the model is trained to learn the underlying structure of the data.

This raises two questions.

1. Shall we give up on neural networks then?
2. But, wait, how does that even work?!



Reason #1: Limits

Reason #1: Neural networks are limited



$+ .007 \times$



$=$



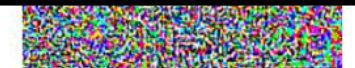
- That at minimum, the purpose of the model is to learn the underlying structure of the data.
- For image classification, the model is trained to learn the underlying structure of the data.

This raises two questions.

1. Shall we give up on neural networks then?

No, because of reason #2, defense (more on this later).

2. But, wait, how does that even work?!



Reason #1: Limits

Reason #1: Neural networks are limited



$+ .007 \times$



$=$



This raises two questions.

1. Shall we give up on neural networks then?
No, because of reason #2, defense (more on this later).
2. **But, wait, how does that even work?!
(And what does this teach us about Neural Networks?)**

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - **Dataset and Dataloader**

Dataset and Dataloader

```
1 # Transform definition
2 # (Basic: only convert image to torch tensor)
3 tf = transforms.Compose([transforms.ToTensor()])
```

```
1 # MNIST dataset and dataloader
2 # (For testing only, we will use a pre-trained model)
3 ds = datasets.MNIST('./data', train = False, \
4                     download = True, transform = tf)
5 test_loader = torch.utils.data.DataLoader(ds, batch_size = 1, \
6                                           shuffle = True)
```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - **Model**

```

1  # Model definition
2  class Net(nn.Module):
3
4      def __init__(self):
5          super(Net, self).__init__()
6          # Conv. 1
7          self.conv1 = nn.Conv2d(1, 10, kernel_size = 5)
8          # Conv. 2
9          self.conv2 = nn.Conv2d(10, 20, kernel_size = 5)
10         # Dropout for Conv. layers
11         self.conv2_drop = nn.Dropout2d()
12         # FC 1
13         self.fc1 = nn.Linear(320, 50)
14         # FC 2
15         self.fc2 = nn.Linear(50, 10)
16
17     def forward(self, x):
18         # Conv. 1 + ReLU + Dropout
19         x = F.relu(F.max_pool2d(self.conv1(x), 2))
20         # Conv. 2 + ReLU + Dropout
21         x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
22         # Flatten
23         x = x.view(-1, 320)
24         # FC 1 + ReLU + Dropout
25         x = F.relu(self.fc1(x))
26         x = F.dropout(x, training = self.training)
27         # FC 2 + Log-Softmax
28         x = self.fc2(x)
29         return F.log_softmax(x, dim = 1)

```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - **(Pre-trained) Model**

```

1  # Model definition
2  class Net(nn.Module):
3
4      def __init__(self):
5          super(Net, self).__init__()
6          # Conv. 1
7          self.conv1 = nn.Conv2d(1, 10, kernel_size = 5)
8          # Conv. 2
9          self.conv2 = nn.Conv2d(10, 20, kernel_size = 5)
10         # Dropout for Conv. layers
11         self.conv2_drop = nn.Dropout2d()
12         # FC 1
13         self.fc1 = nn.Linear(320, 50)
14         # FC 2
15         self.fc2 = nn.Linear(50, 10)
16
17     def forward(self, x):
18         # Conv. 1 + ReLU + Dropout
19         x = F.relu(F.max_pool2d(self.conv1(x), 2))
20         # Conv. 2 + ReLU + Dropout
21         x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
22         # Flatten
23         x = x.view(-1, 320)
24         # FC 1 + ReLU + Dropout
25         x = F.relu(self.fc1(x))
26         x = F.dropout(x, training = self.training)
27         # FC 2 + Log-Softmax
28         x = self.fc2(x)
29         return F.log_softmax(x, dim = 1)

```

```

1  # Load the pretrained model
2  model = Net().to(device)
3  pretrained_model = "./mnist_model.data"
4  model.load_state_dict(torch.load(pretrained_model, \
5                                  map_location = 'cpu'))

```

<All keys matched successfully>

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - **Attack function**

```
1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [- epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[2*(np.random.random() - 0.5)*epsilon
8                                for i in range(img_rows)]
9                                for j in range(img_cols)])])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image
```

Our first attack: Epsilon Noising

Definition (Epsilon Noising Method):

The **Epsilon Noising Method (ENM)** is the simplest type of attack. It consists of generating an image $\tilde{x}$, by **adding a random noise to each pixel of an image x , with amplitude $[-\epsilon, \epsilon]$.**

$$\forall i, j \in \text{Pixel_Indexes}_x$$

$$\tilde{x}_{i,j} = x_{i,j} + \omega_{i,j}$$

$$\begin{cases} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (\text{Unif. Dist.}) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (\text{Normal Dist.}) \end{cases}$$

```

1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [- epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[2*(np.random.random() - 0.5)*epsilon
8                                for i in range(img_rows)]
9                                for j in range(img_cols)]])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Generate noise vector
with same size as
image, and amplitude
in $[-\epsilon, \epsilon]$.

Our first attack: Epsilon Noising

Definition (Epsilon Noising Method):

The **Epsilon Noising Method (ENM)** is the simplest type of attack. It consists of generating an image $\tilde{x}$, by adding a random noise to each pixel of an image x , with amplitude $[-\epsilon, \epsilon]$.

```

1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [-epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[2*(np.random.random() - 0.5)*epsilon
8                                for i in range(img_rows)]
9                                for j in range(img_cols)]])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```

$$\forall i, j \in \text{Pixel_Indexes}_x$$

$$\tilde{x}_{i,j} = x_{i,j} + \omega_{i,j}$$

$$\begin{cases} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (\text{Unif. Dist.}) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (\text{Normal Dist.}) \end{cases}$$



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Our first attack: Epsilon Noising

Add noise to original image.

Definition (Epsilon Noising Method):

The **Epsilon Noising Method (ENM)** is the simplest type of attack. It consists of generating an image $\tilde{x}$, by adding a random noise to each pixel of an image x , with amplitude $[-\epsilon, \epsilon]$.

```

1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [-epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[2*(np.random.random() - 0.5)*epsilon
8                               for i in range(img_rows)]
9                               for j in range(img_cols)])])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```

$$\forall i, j \in \text{Pixel_Indexes}_x$$

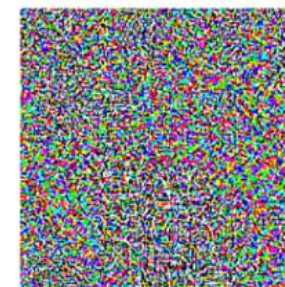
$$\tilde{x}_{i,j} = x_{i,j} + \omega_{i,j}$$

$$\begin{cases} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (\text{Unif. Dist.}) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (\text{Normal Dist.}) \end{cases}$$



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Our first attack: Epsilon Noising

Clipping to prevent unwanted pixel values.

Reminder (**Clipping** a value):

Clipping a value x forces it to remain in an interval $[a, b]$, with $a \leq b$.

We define the **clipping function** $\gamma_{a,b}(x)$, as follows.

$$\gamma_{a,b}(x) = \max(a, \min(x, b))$$

$$\gamma_{a,b}(x) = \begin{cases} a & \text{if } x \leq a \\ b & \text{if } x \geq b \\ x & \text{otherwise} \end{cases}$$

```

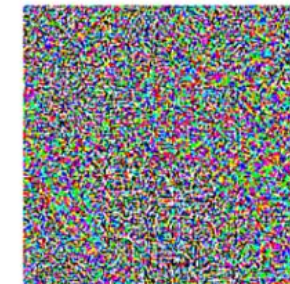
1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [-epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[[2*(np.random.random() - 0.5)*epsilon
8                                for i in range(img_rows)]
9                                for j in range(img_cols)]]])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Our first attack: Epsilon Noising

Clipping to prevent pixel values to go out of [0, 1].
(Normalization taken into account)

Definition (Epsilon Noising Method):

The **Epsilon Noising Method (ENM)** is the simplest type of attack. It consists of generating an image $\tilde{x}$, by adding a random noise to each pixel of an image x , with amplitude $[-\epsilon, \epsilon]$.

```

1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [-epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[[2*(np.random.random() - 0.5)*epsilon
8                                for i in range(img_rows)]
9                                for j in range(img_cols)]]])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```

$$\forall i, j \in \text{Pixel_Indexes}_x$$

$$\tilde{x}_{i,j} = \gamma_{0,1}(x_{i,j} + \omega_{i,j})$$

$$\begin{cases} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (\text{Unif. Dist.}) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (\text{Normal Dist.}) \end{cases}$$



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model

```

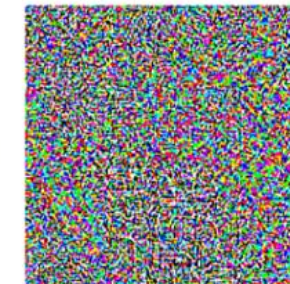
1 def enm_attack(image, epsilon):
2
3     # Generate noise matrix, with same shape as image,
4     # and random values in [- epsilon, epsilon]
5     img_rows = image.shape[-2]
6     img_cols = image.shape[-1]
7     epsilon_mat = np.asarray([[[2*(np.random.random() - 0.5)*epsilon
8                               for i in range(img_rows)]
9                               for j in range(img_cols)])])
10
11     # Create the attack image by adjusting each pixel of the input image
12     eps_image = image.detach().numpy() + epsilon_mat
13
14     # Clipping eps_image to maintain pixel values into the [0, 1] range
15     eps_image = torch.from_numpy(eps_image).float()
16     eps_image = torch.clamp(eps_image, 0, 1)
17
18     # Return
19     return eps_image

```



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Restricted

Testing Function

This function attempts to 1) track how an attack with amplitude epsilon, used on the samples in the test_loader will affect our model;

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67        # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Restricted

Testing Function

This function attempts to 1) track how an attack with amplitude epsilon, used on the samples in the test_loader will affect our model; and 2) return attack samples that worked for later visualization.

```

1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()

```

```

34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if epsilon == 0 and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list

```

Testing Function

Restricted
This will track how many samples were correctly classified despite the attack being performed on the test samples.

```
1 def test(model, device, test_loader, epsilon):
```

```
2     # Counter for correct values (used for accuracy)  
3     correct_counter = 0
```

```
4  
5     # List of successful adversarial samples  
6     adv_examples_list = []
```

```
7  
8  
9     # Loop over all examples in test set  
10    for image, label in test_loader:
```

```
11  
12        # Send the data and label to the device  
13        image, label = image.to(device), label.to(device)
```

```
14  
15        # Pass the image through the model  
16        output = model(image)  
17        # Get the index of the max log-probability  
18        init_pred = output.max(1, keepdim = True)[1]
```

```
19  
20        # If the initial prediction is wrong, do not  
21        # bother attacking, skip current image  
22        if init_pred.item() != label.item():  
23            continue
```

```
24  
25        # Calculate the loss  
26        loss = F.nll_loss(output, label)
```

```
27  
28        # Zero all existing gradients  
29        model.zero_grad()
```

```
30  
31        # Backpropagate  
32        loss.backward()
```

```
34    # Call ENM Attack
```

```
35    eps_image = enm_attack(image, epsilon)
```

```
36  
37    # Re-classify the epsilon image
```

```
38    output2 = model(eps_image)
```

```
39    # Get the index of the max log-probability
```

```
40    eps_pred = output2.max(1, keepdim = True)[1]
```

```
41  
42    # Check for successful attack
```

```
43    # (Successful meaning eps_pred label different from init_pred)
```

```
44    if eps_pred.item() == label.item():
```

```
45        correct_counter += 1
```

```
46        # Special case for saving 0 epsilon examples
```

```
47        # (Maximal number of saved samples is set to 5)
```

```
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
```

```
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
```

```
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
```

```
51    else:
```

```
52        # Save some adv examples for visualization later
```

```
53        # (Maximal number of saved samples is set to 5)
```

```
54        if len(adv_examples_list) < 5:
```

```
55            adv_ex = eps_image.squeeze().detach().cpu().numpy()
```

```
56            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
```

```
57  
58    # Calculate final accuracy for this epsilon value
```

```
59    final_acc = correct_counter/float(len(test_loader))
```

```
60  
61    # Display for progress
```

```
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \  
63                                                                correct_counter, \  
64                                                                len(test_loader), \  
65                                                                final_acc))
```

```
66  
67    # Return the accuracy and an adversarial example
```

```
68    return final_acc, adv_examples_list
```

Testing Function

Restricted
This will store up to 5 attack samples that made the model malfunction (used for visualization later).

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51            else:
52                # Save some adv examples for visualization later
53                # (Maximal number of saved samples is set to 5)
54                if len(adv_examples_list) < 5:
55                    adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                    adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67        # Return the accuracy and an adversarial example
68        return final_acc, adv_examples_list
```

Testing Function

Restricted

This is very typical for our test functions so far, just browsing through (normal) test samples and trying those on our model.

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67        # Return the accuracy and an adversarial example
68        return final_acc, adv_examples_list
```

Testing Function

Restricted
If the model already misclassifies the sample, do not bother attacking
(Attack could make the model right!).

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
```

```
34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51    else:
52        # Save some adv examples for visualization later
53        # (Maximal number of saved samples is set to 5)
54        if len(adv_examples_list) < 5:
55            adv_ex = eps_image.squeeze().detach().cpu().numpy()
56            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Testing Function

Restricted

This is again very typical. Right now, it does not appear necessary, but more advanced attacks will rely on the gradients of the model, and that would be the way to compute them (more on this later!).

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
```

```
34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Restricted

Restricted

Generate an attack sample, using our ENM attack function.



Testing Function

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
```

```
34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Restricted

Restricted

Try attack sample on our model.



Testing Function

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
```

```
34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Restricted

Testing Function

Restricted
If attack sample is correctly classified, the attack is a failure.
Increase correct score by one.

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                    correct_counter, \
64                                                                    len(test_loader), \
65                                                                    final_acc))
66
67        # Return the accuracy and an adversarial example
68        return final_acc, adv_examples_list
```

Testing Function

Add sample to adversarial samples list if epsilon = 0 and list not full
(attacking with epsilon = 0 will always fail as it will not modify the image).
This gives the baseline accuracy of the model before attacks.

```

1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51            else:
52                # Save some adv examples for visualization later
53                # (Maximal number of saved samples is set to 5)
54                if len(adv_examples_list) < 5:
55                    adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                    adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                    correct_counter, \
64                                                                    len(test_loader), \
65                                                                    final_acc))
66
67        # Return the accuracy and an adversarial example
68        return final_acc, adv_examples_list

```

Testing Function

Restricted

If attack sample makes the model misclassify, it is a successful attack.
Do not increase correct_counter, and store sample in adversarial samples list if not already full.

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
```

```
34    # Call ENM Attack
35    eps_image = enm_attack(image, epsilon)
36
37    # Re-classify the epsilon image
38    output2 = model(eps_image)
39    # Get the index of the max log-probability
40    eps_pred = output2.max(1, keepdim = True)[1]
41
42    # Check for successful attack
43    # (Successful meaning eps_pred label different from init_pred)
44    if eps_pred.item() == label.item():
45        correct_counter += 1
46        # Special case for saving 0 epsilon examples
47        # (Maximal number of saved samples is set to 5)
48        if (epsilon == 0) and (len(adv_examples_list) < 5):
49            adv_ex = eps_image.squeeze().detach().cpu().numpy()
50            adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58    # Calculate final accuracy for this epsilon value
59    final_acc = correct_counter/float(len(test_loader))
60
61    # Display for progress
62    print("Epsilon: {} - Test Accuracy = {}/{}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67    # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Restricted

Testing Function

Restricted
After the for loop, compute accuracy of model after attack.

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67        # Return the accuracy and an adversarial example
68    return final_acc, adv_examples_list
```

Testing Function

Restricted

Display accuracy for given epsilon value.

Return accuracy score and the list of five adversarial samples.

```
1 def test(model, device, test_loader, epsilon):
2
3     # Counter for correct values (used for accuracy)
4     correct_counter = 0
5
6     # List of successful adversarial samples
7     adv_examples_list = []
8
9     # Loop over all examples in test set
10    for image, label in test_loader:
11
12        # Send the data and label to the device
13        image, label = image.to(device), label.to(device)
14
15        # Pass the image through the model
16        output = model(image)
17        # Get the index of the max log-probability
18        init_pred = output.max(1, keepdim = True)[1]
19
20        # If the initial prediction is wrong, do not
21        # bother attacking, skip current image
22        if init_pred.item() != label.item():
23            continue
24
25        # Calculate the loss
26        loss = F.nll_loss(output, label)
27
28        # Zero all existing gradients
29        model.zero_grad()
30
31        # Backpropagate
32        loss.backward()
33
34        # Call ENM Attack
35        eps_image = enm_attack(image, epsilon)
36
37        # Re-classify the epsilon image
38        output2 = model(eps_image)
39        # Get the index of the max log-probability
40        eps_pred = output2.max(1, keepdim = True)[1]
41
42        # Check for successful attack
43        # (Successful meaning eps_pred label different from init_pred)
44        if eps_pred.item() == label.item():
45            correct_counter += 1
46            # Special case for saving 0 epsilon examples
47            # (Maximal number of saved samples is set to 5)
48            if (epsilon == 0) and (len(adv_examples_list) < 5):
49                adv_ex = eps_image.squeeze().detach().cpu().numpy()
50                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
51        else:
52            # Save some adv examples for visualization later
53            # (Maximal number of saved samples is set to 5)
54            if len(adv_examples_list) < 5:
55                adv_ex = eps_image.squeeze().detach().cpu().numpy()
56                adv_examples_list.append((init_pred.item(), eps_pred.item(), adv_ex))
57
58        # Calculate final accuracy for this epsilon value
59        final_acc = correct_counter/float(len(test_loader))
60
61        # Display for progress
62        print("Epsilon: {} - Test Accuracy = {}/{} = {}".format(epsilon, \
63                                                                correct_counter, \
64                                                                len(test_loader), \
65                                                                final_acc))
66
67        # Return the accuracy and an adversarial example
68        return final_acc, adv_examples_list
```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - **Testing effect of attack on model**

```
1  epsilons = [0, .1, .2, .5, 1, 2, 5, 10]
2  accuracies = []
3  examples = []
4
5  # Run test() function for each epsilon
6  for eps in epsilons:
7      acc, ex = test(model, device, test_loader, eps)
8      accuracies.append(acc)
9      examples.append(ex)
```

```
Epsilon: 0 - Test Accuracy = 9810/10000 = 0.981
Epsilon: 0.1 - Test Accuracy = 9792/10000 = 0.9792
Epsilon: 0.2 - Test Accuracy = 9775/10000 = 0.9775
Epsilon: 0.5 - Test Accuracy = 9578/10000 = 0.9578
Epsilon: 1 - Test Accuracy = 6367/10000 = 0.6367
Epsilon: 2 - Test Accuracy = 2203/10000 = 0.2203
Epsilon: 5 - Test Accuracy = 1162/10000 = 0.1162
Epsilon: 10 - Test Accuracy = 1074/10000 = 0.1074
```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model
 - **Accuracy drop and attack samples visualization**

Display a simple plot of accuracy vs. epsilon value for our given attack and model.

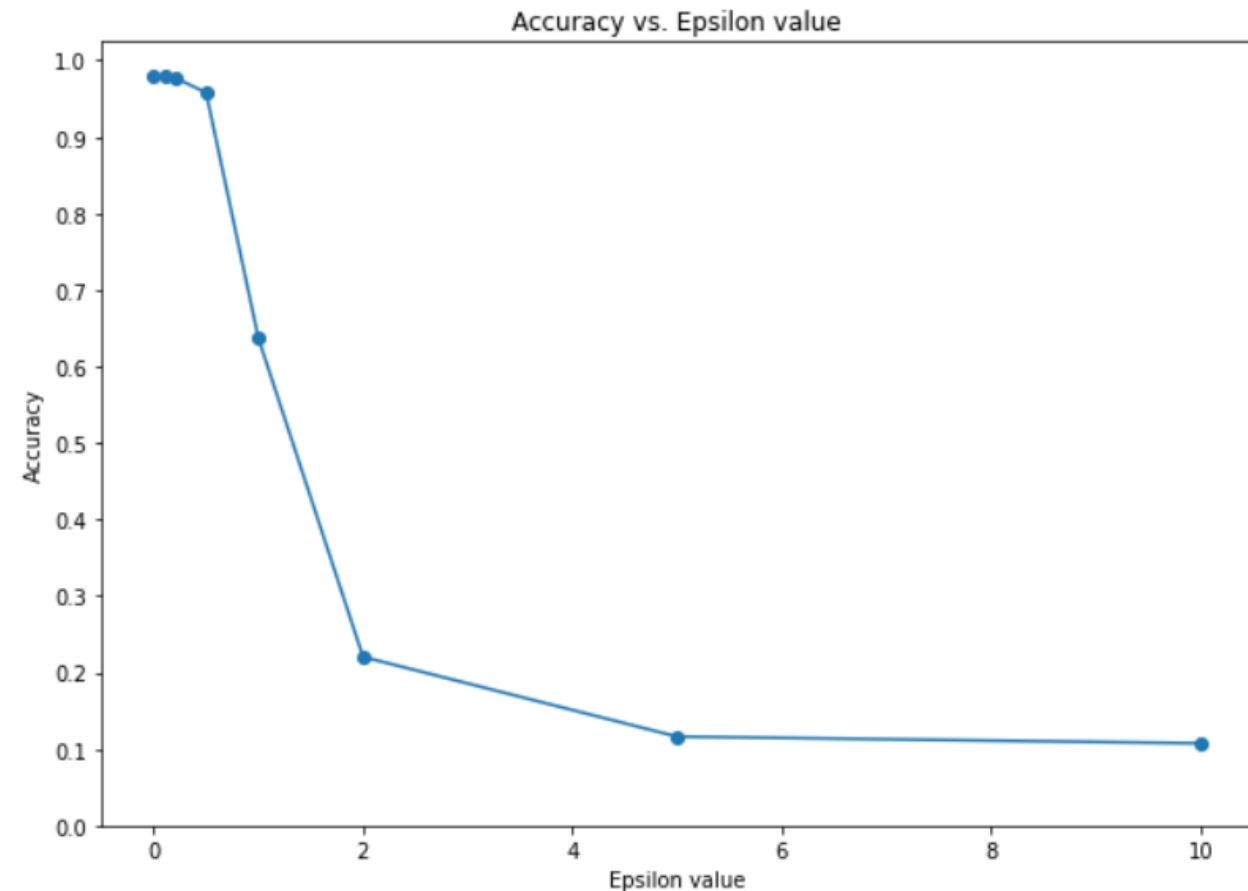
```
1  # Initialize figure
2  plt.figure(figsize = (7, 10))
3
4  # Display accuracy vs. Epsilon values plot
5  plt.plot(epsilons, accuracies, "o-")
6
7  # Adjust x-axis and y-axis labels and ticks
8  plt.yticks(np.arange(0, 1.1, step = 0.1))
9  #plt.xticks(np.arange(0, .35, step = 0.05))
10 plt.title("Accuracy vs. Epsilon value")
11 plt.xlabel("Epsilon value")
12 plt.ylabel("Accuracy")
13
14 # Display
15 plt.show()
```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model
 - **Accuracy drop and attack samples visualization**

Display a simple plot of accuracy vs. epsilon value for our given attack and model.



Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model
 - **Accuracy drop and attack samples visualization**

Display some adversarial samples for each value of epsilon.

```

1  # Plot several examples of adversarial samples at each epsilon
2  cnt = 0
3
4  # Initialize figure
5  plt.figure(figsize = (10, 10))
6
7  # Browse through epsilon values and adversarial examples
8  for i in range(len(epsilons)):
9      for j in range(len(examples[i])):
10         cnt += 1
11         plt.subplot(len(epsilons), len(examples[0]), cnt)
12
13         # Remove x-axis and y-axis ticks from plot
14         plt.xticks([], [])
15         plt.yticks([], [])
16
17         # Labels for y axis
18         if j == 0:
19             plt.ylabel("Eps: {}".format(epsilons[i]), fontsize = 14)
20
21         # Labels for each image subplot
22         orig, adv, ex = examples[i][j]
23         plt.title("{} -> {}".format(orig, adv))
24
25         # Display image
26         plt.imshow(ex, cmap = "gray")
27
28 # Display full plot
29 plt.tight_layout()
30 plt.show()

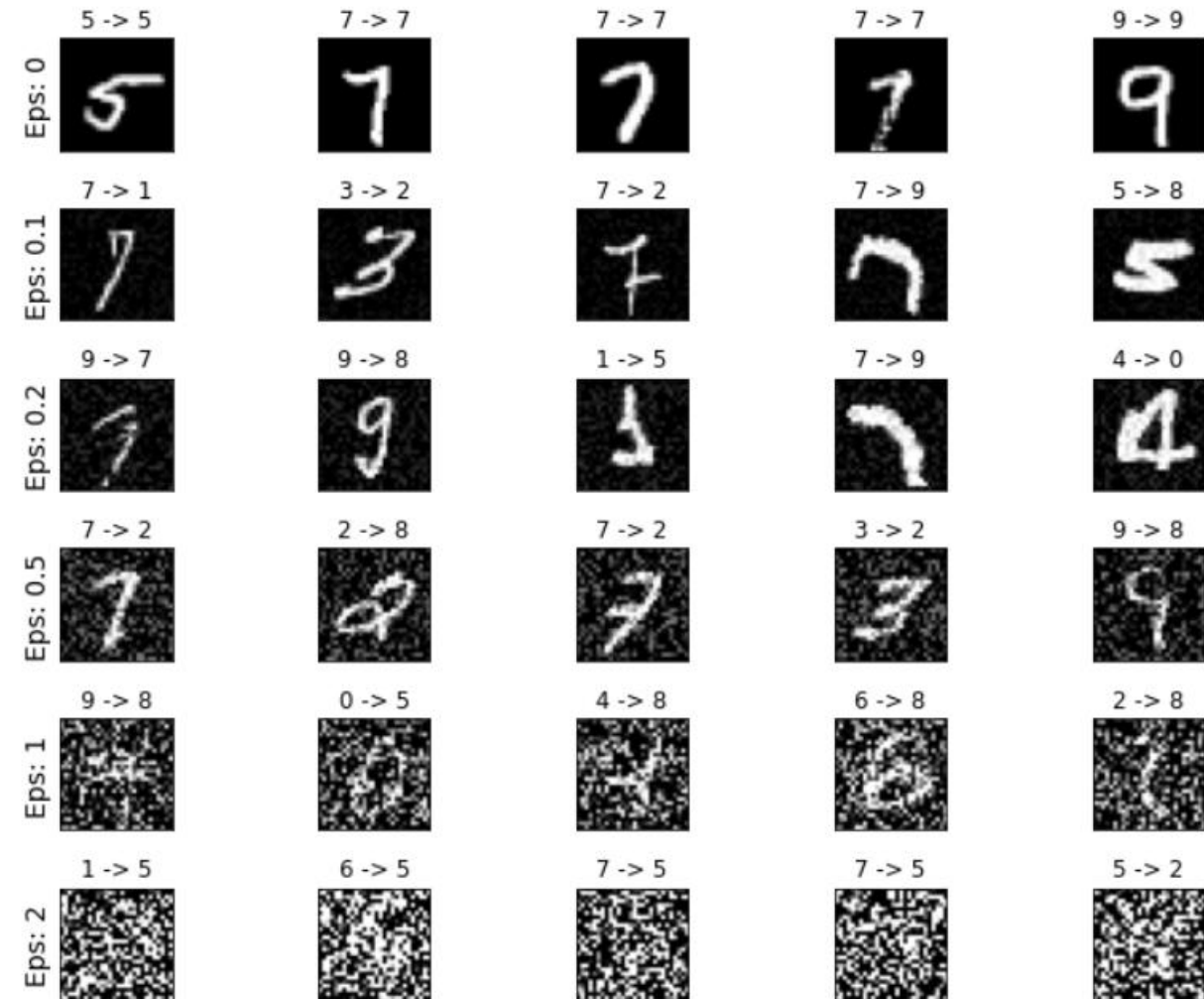
```

Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model
 - **Accuracy drop and attack samples visualization**

Display some adversarial samples for each value of epsilon.



Notebooks structure

Please refer to Notebook 1. Using Epsilon Noising Attack to Generate Attack Samples.

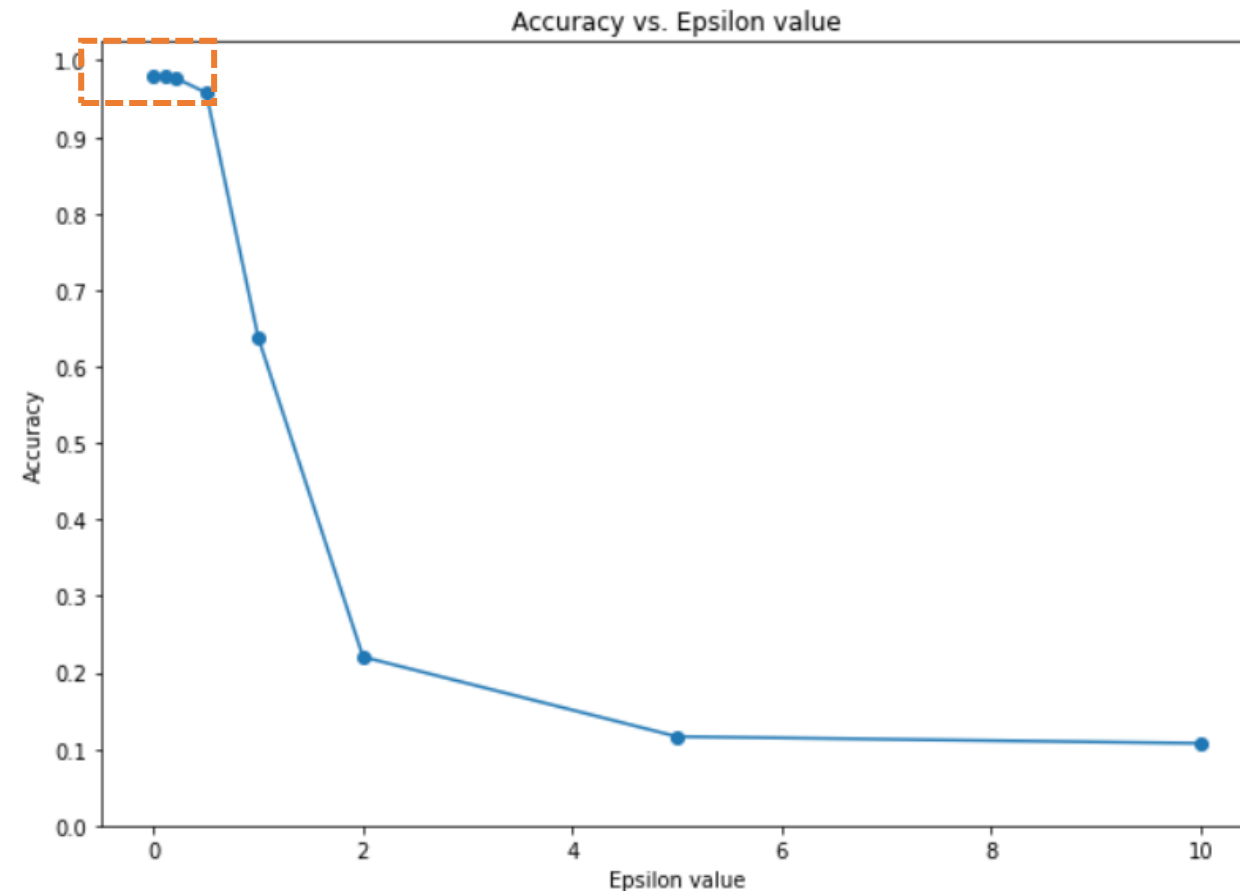
- All notebooks this week follow the same structure
 - Dataset and Dataloader
 - (Pre-trained) Model
 - Attack function
 - Testing effect of attack on model
 - Accuracy drop and attack samples visualization
 - ~~Defense against such an attack~~

SOMETHING
FOR LATER...

Effect of ENM on accuracy

Adding noise to an image tends to make the model malfunction.

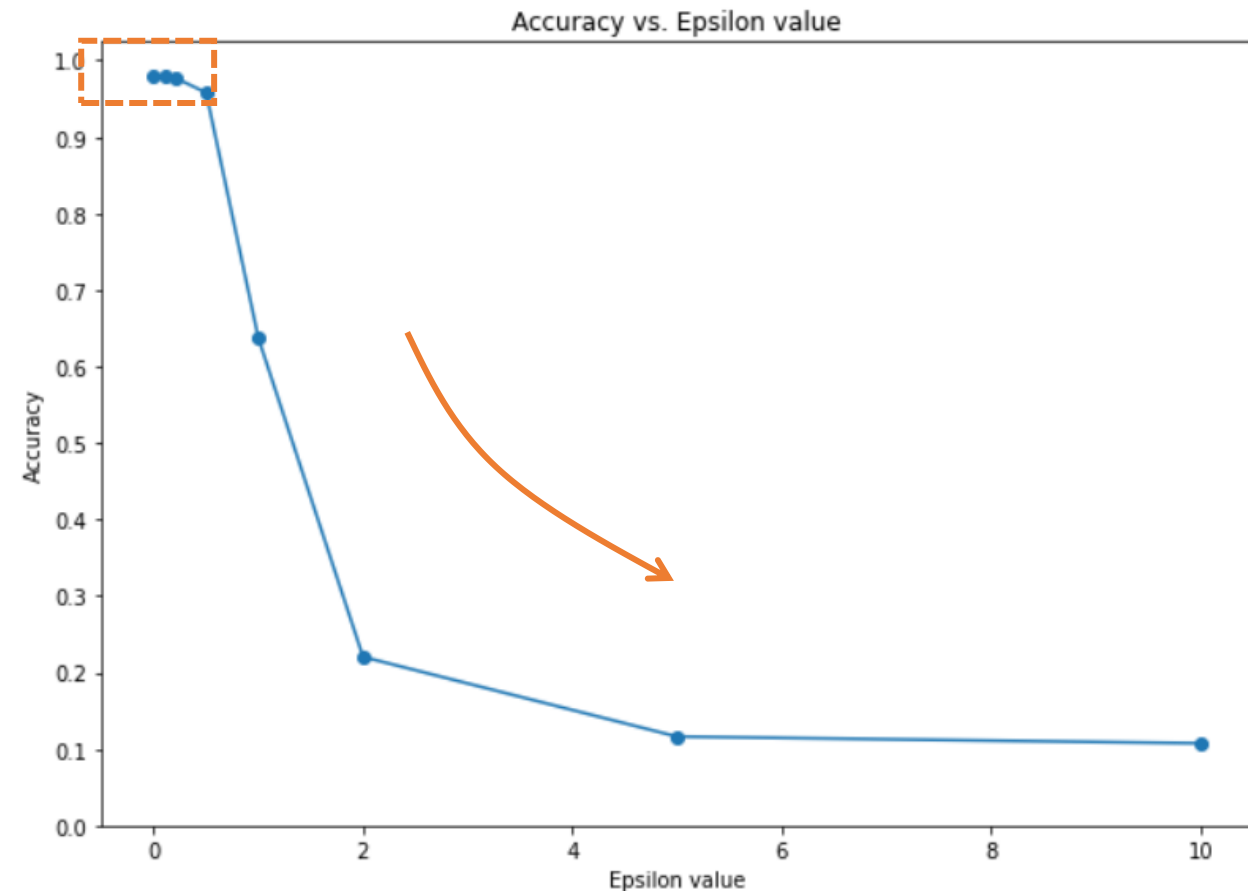
- When no noise (epsilon = 0), 98.1% accuracy.
 - This is our **baseline accuracy**.



Effect of ENM on accuracy

Adding noise to an image tends to make the model malfunction.

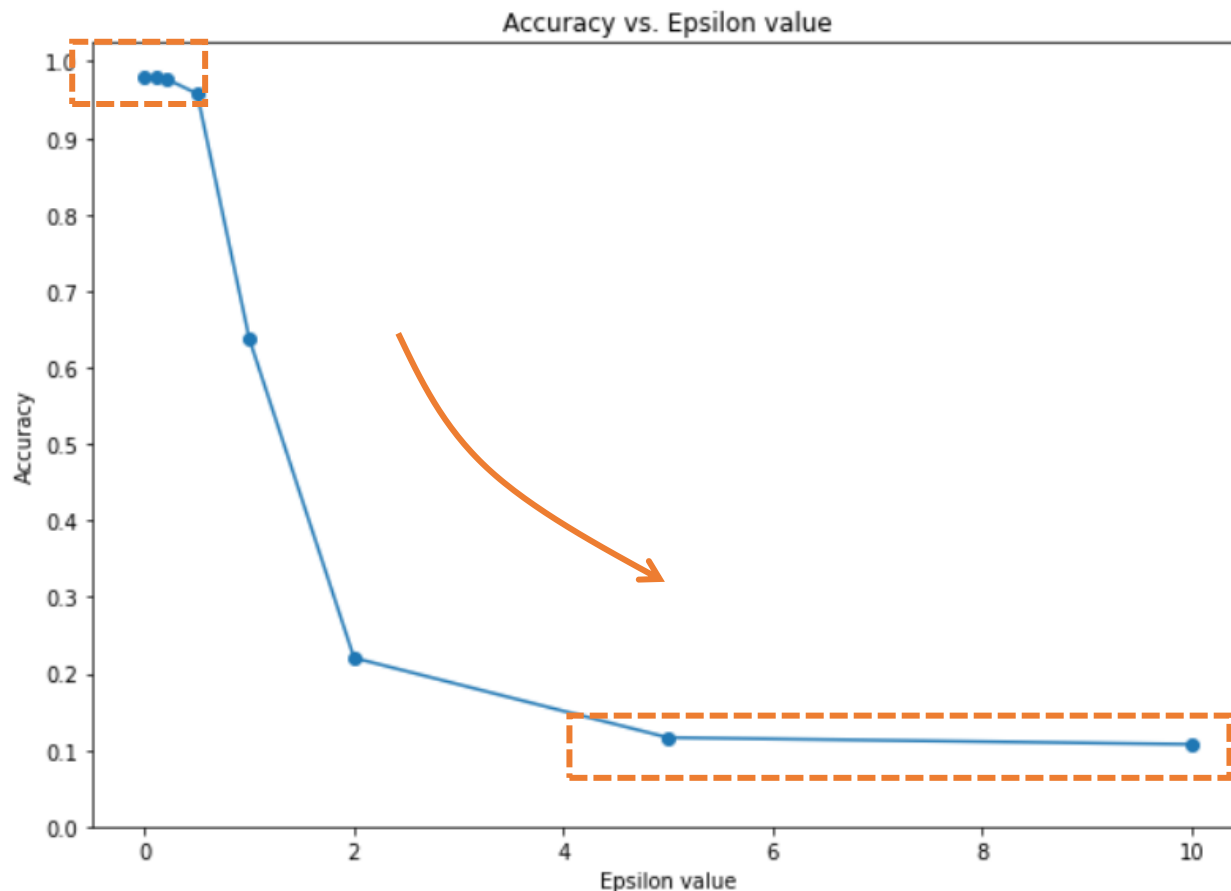
- When no noise (epsilon = 0), 98.1% accuracy.
 - This is our **baseline accuracy**.
- Accuracy decreases, the further we increment the noise amplitude epsilon.



Effect of ENM on accuracy

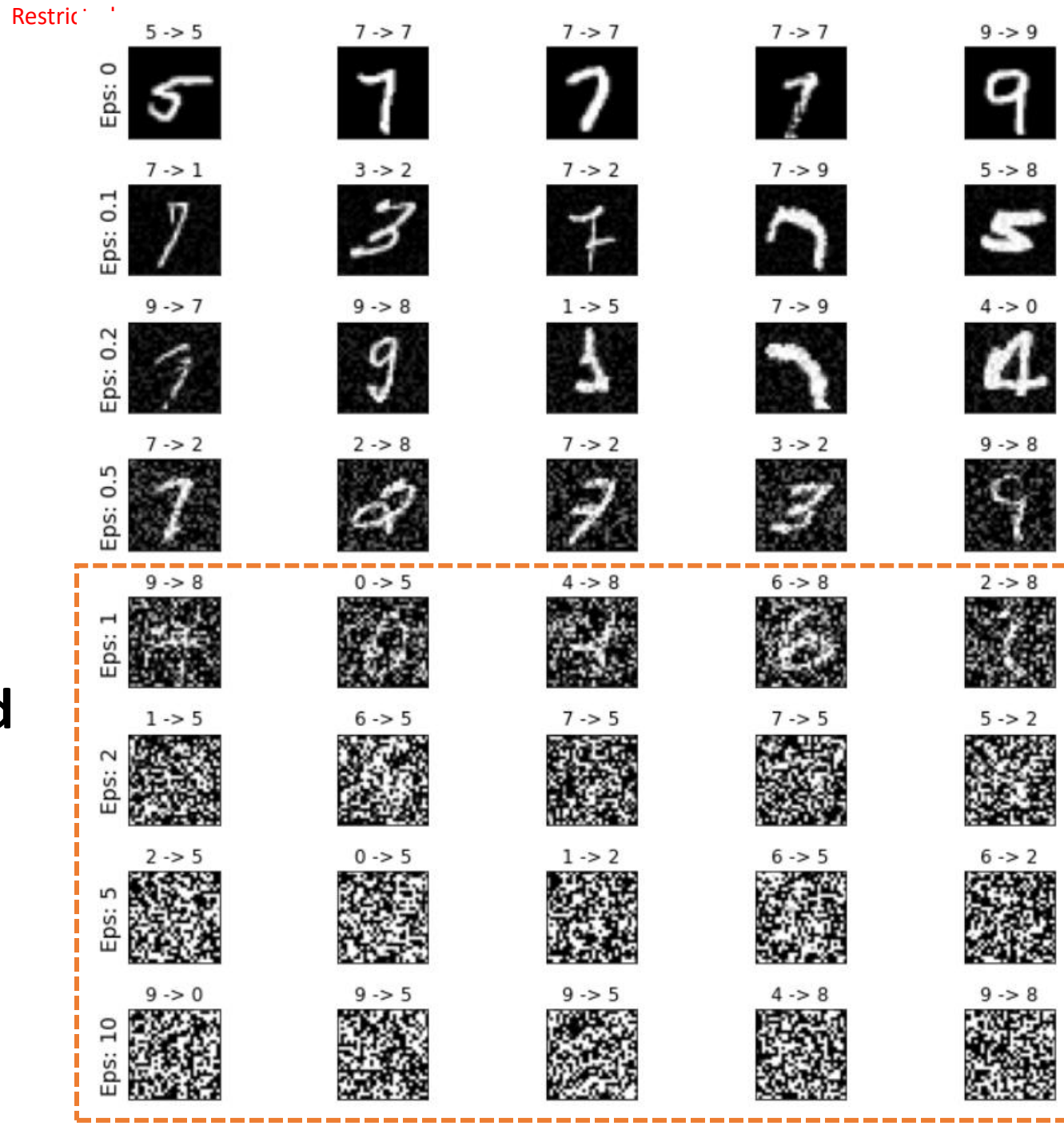
Adding noise to an image tends to make the model malfunction.

- When no noise (epsilon = 0), 98.1% accuracy.
 - This is our **baseline accuracy**.
- Accuracy decreases, the further we increment the noise amplitude epsilon.
- Eventually, with full noise (large epsilon), the image will be **randomly classified** (Accuracy $\sim 10\%$).



What makes a good attack sample?

- However, for large values of epsilon, the attack samples **simply become random noise**.
- This is mostly why the classifier ends up struggling.
- These are **NOT considered good attack samples!**
- (**Think:** humans would struggle to classify those as well!)

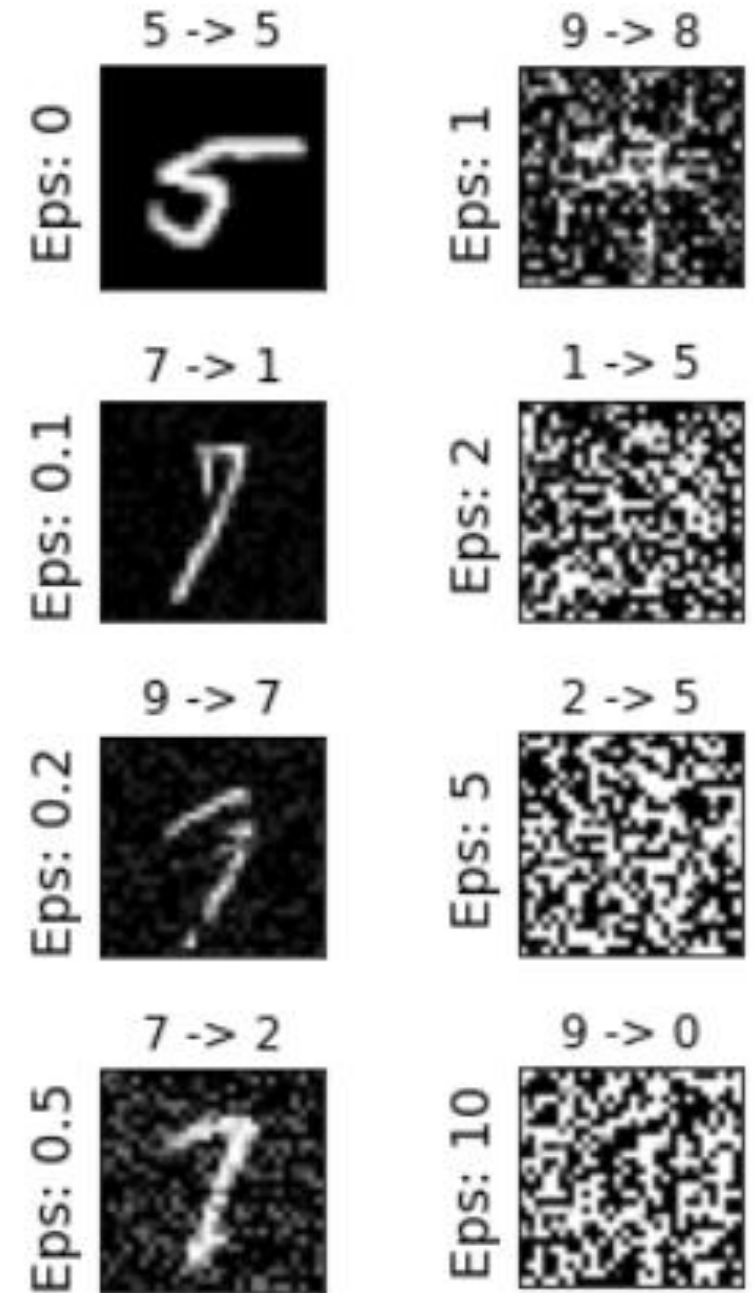


What makes a good attack sample?

Definition (what makes a “good” attack sample):

A “good” attack sample satisfies two properties:

1. **Model failure:** it makes the model malfunction.
2. **Plausibility:** It looks “plausible” or “normal” to a human. Meaning that it looks like it could have been a sample from the dataset.

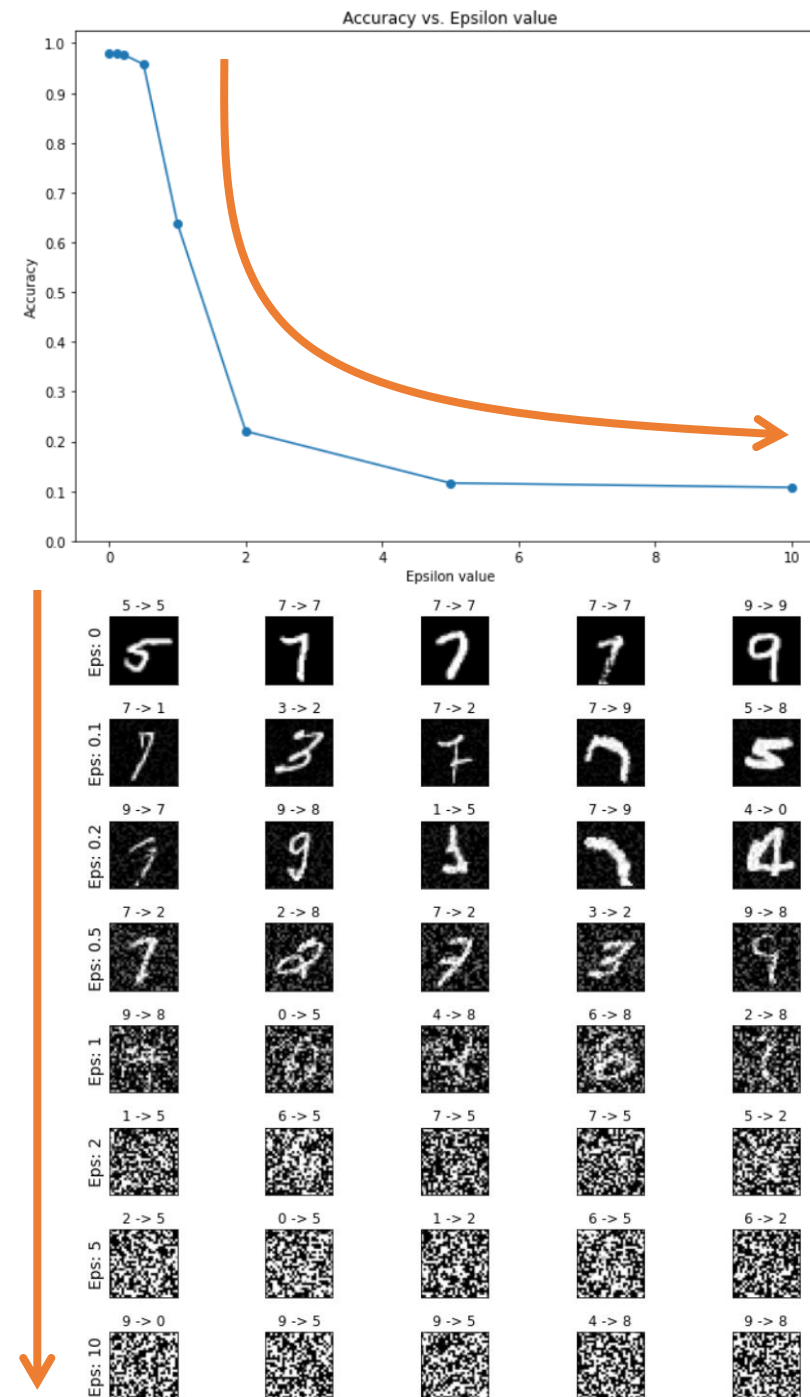
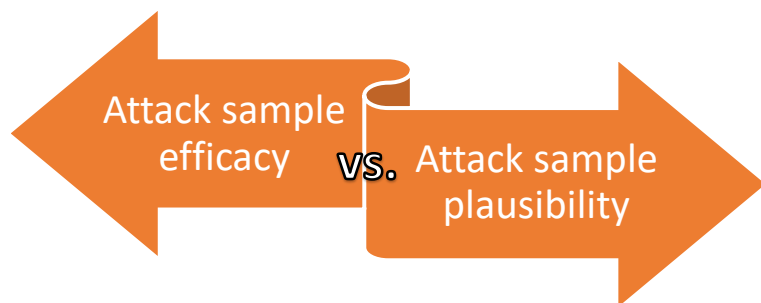


Attack samples tradeoff

Definition (attack samples tradeoff in adversarial ML):

Attack samples are subject to a **tradeoff**. In general,

- the higher the odds of the attack sample to make the model malfunction,
- the less plausible it will look.



Plausibility

In general, we would like to have the generated attack sample $\tilde{x}$, to be “**close enough**” to the original sample x .

- This is a simple way to ensure **plausibility** for the attack sample.

$$\begin{aligned} & \forall i, j \in Pixel_Indexes_x \\ & \quad \tilde{x}_{i,j} = x_{i,j} + \omega_{i,j} \\ & \left\{ \begin{array}{ll} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (Unif. Dist.) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (Normal Dist.) \end{array} \right. \end{aligned}$$

Plausibility

In general, we would like to have the generated attack sample $\tilde{x}$, to be “**close enough**” to the original sample x .

- This is a simple way to ensure **plausibility** for the attack sample.
- In practice, we often enforce a **constraint** on a **distance metric** (or norm) between both the **original image** x and **attack sample** $\tilde{x}$.

$$\begin{aligned} & \forall i, j \in Pixel_Indexes_x \\ & \tilde{x}_{i,j} = x_{i,j} + \omega_{i,j} \\ & \left\{ \begin{array}{ll} \omega_{i,j} \rightarrow U([- \epsilon, \epsilon]) & (Unif. Dist.) \\ \omega_{i,j} \rightarrow N([- \epsilon, \epsilon]) & (Normal Dist.) \end{array} \right. \end{aligned}$$

$$\|\tilde{x} - x\| \leq \alpha,$$

with α chosen arbitrarily

A reminder about norms

- **L^0 norm:** bounds the total number of pixels in $\tilde{x}$ that can be modified with respect to x (though they can be modified by any amount).

$$\|\tilde{x} - x\|_0 = \text{card}(\{(i, j) \text{ s.t. } x_{i,j} \neq \tilde{x}_{i,j}\})$$

A reminder about norms

- **L^0 norm:** bounds the total number of pixels in $\tilde{x}$ that can be modified with respect to x (though they can be modified by any amount).
- **L^1 norm:** bounds the average absolute distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x .

$$\|\tilde{x} - x\|_1 = \frac{1}{N} \sum_{i,j} |\tilde{x}_{i,j} - x_{i,j}|, \quad \text{with } N \text{ the number of pixels}$$

A reminder about norms

- **L^0 norm:** bounds the total number of pixels in $\tilde{x}$ that can be modified with respect to x (though they can be modified by any amount).
- **L^1 norm:** bounds the average absolute distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x .
- **L^2 norm:** bounds the total squared distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x . Most commonly referred to as Euclidean distance.

$$\|\tilde{x} - x\|_2 = \frac{1}{N} \sqrt{\sum_{i,j} (\tilde{x}_{i,j} - x_{i,j})^2}, \quad \text{with } N \text{ the number of pixels}$$

A reminder about norms

- **L^0 norm:** bounds the total number of pixels in $\tilde{x}$ that can be modified with respect to x (though they can be modified by any amount).
- **L^1 norm:** bounds the average absolute distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x .
- **L^2 norm:** bounds the total squared distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x . Often referred to as the Euclidean distance.
- **L^∞ norm:** bounds the maximum difference between any pixel in $\tilde{x}$ and the corresponding pixel in x . Often referred to as max norm.

$$\|\tilde{x} - x\|_\infty = \max_{i,j} (|\tilde{x}_{i,j} - x_{i,j}|)$$

A reminder about norms

- **L^0 norm:** bounds the total number of pixels in $\tilde{x}$ that can be modified with respect to x (though they can be modified by any amount).
- **L^1 norm:** bounds the average absolute distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x .
- **L^2 norm:** bounds the total squared distance between the values of pixels in $\tilde{x}$ and the corresponding pixels in x . Often referred to as the Euclidean distance.
- **L^∞ norm:** bounds the maximum difference between any pixel in $\tilde{x}$ and the corresponding pixel in x . Often referred to as max norm.

$$\|\tilde{x} - x\|_\infty = \max_{i,j} (|\tilde{x}_{i,j} - x_{i,j}|)$$

Impact of noise on classifiers

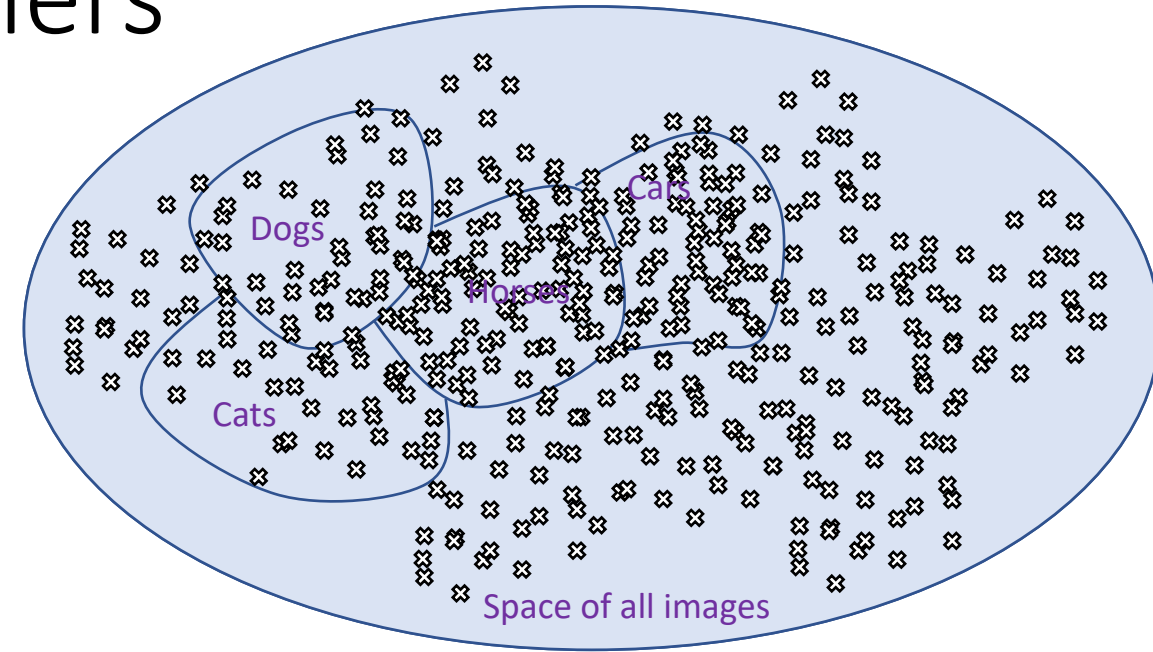
- **Expectation for Neural Networks:** Just like with human perception, small changes on inputs should not yield a different label!
- **Unfortunately, deep learning predictions are different:** Deep learning algorithms process data differently from humans, with strong discontinuities in the change of prediction as a function the inputs.
- **And that is the reason for their vulnerability to attacks.**

Following: A tentative of explaining why that is the case (not the absolute truth, but my intuition as to why this happens!)

Impact of noise on classifiers

Consider the arbitrary representation, with regions and boundaries between them.

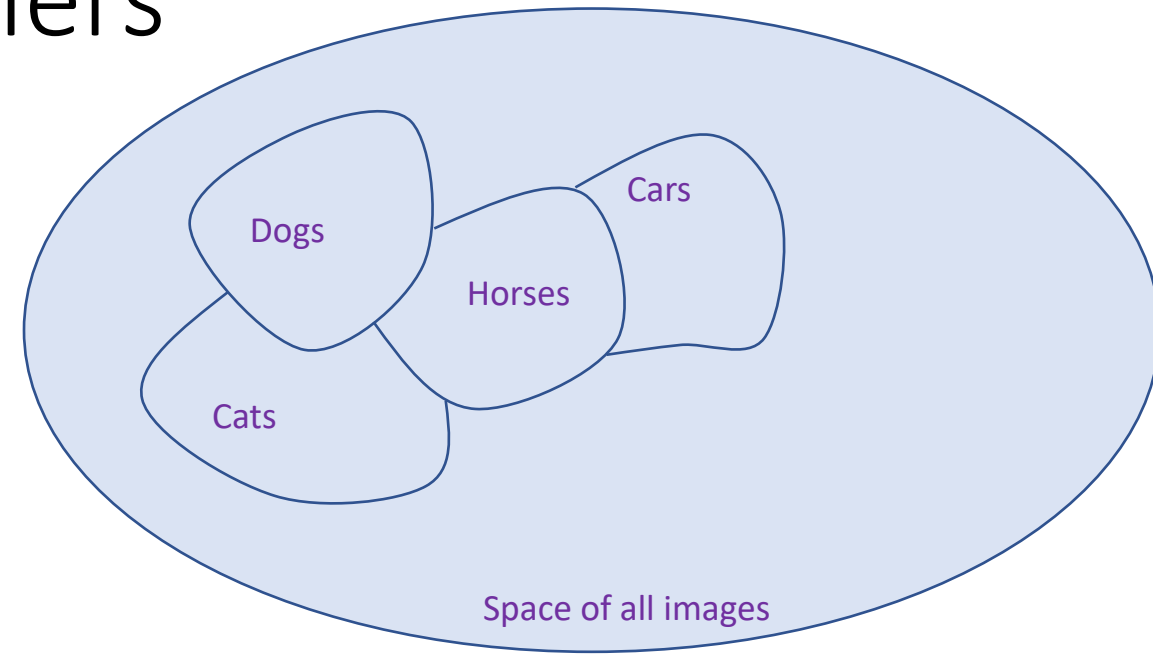
- **Misconception #1:** the whole space of possible inputs was densely filled with training examples during training.



Impact of noise on classifiers

Consider the arbitrary representation, with regions and boundaries between them.

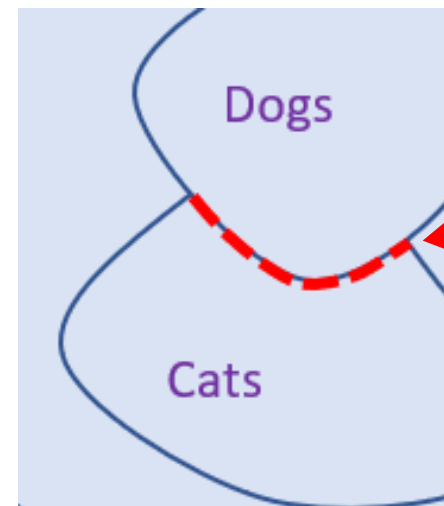
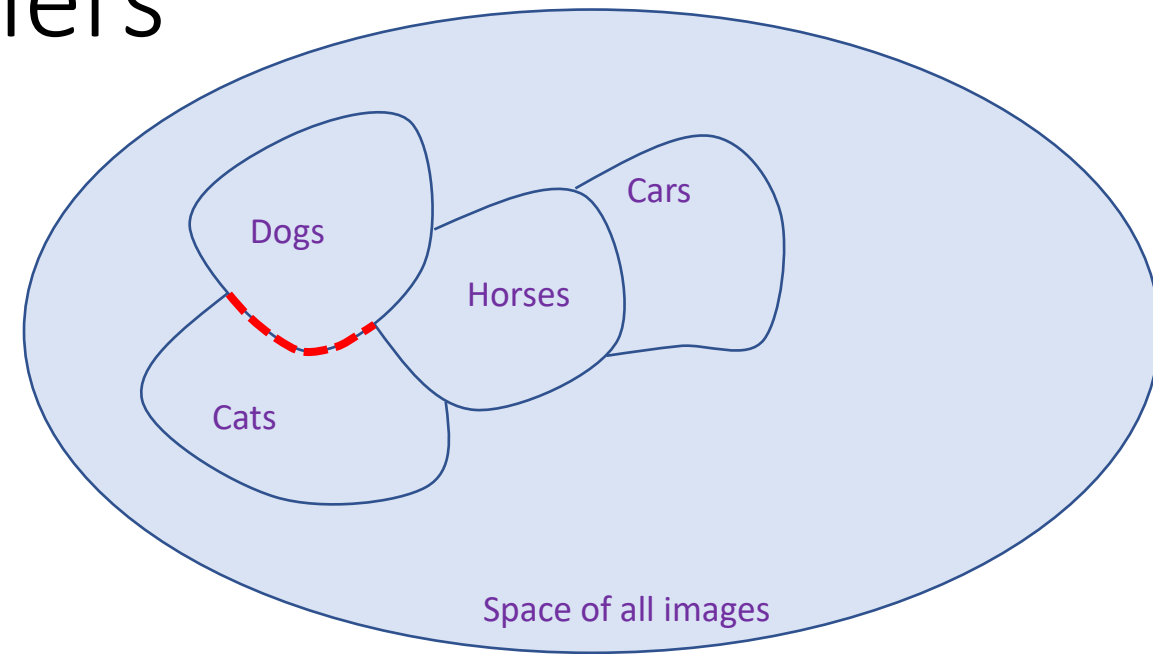
- **Misconception #1:** the whole space of possible inputs was densely filled with training examples during training.
- **Misconception #2:** regions are contiguous and filled with samples.



Impact of noise on classifiers

Consider the arbitrary representation, with regions and boundaries between them.

- **Misconception #1:** the whole space of possible inputs was densely filled with training examples during training.
- **Misconception #2:** regions are contiguous and filled with samples.
- **Misconception #3:** the decision boundaries between classes are smooth and make perfect sense.

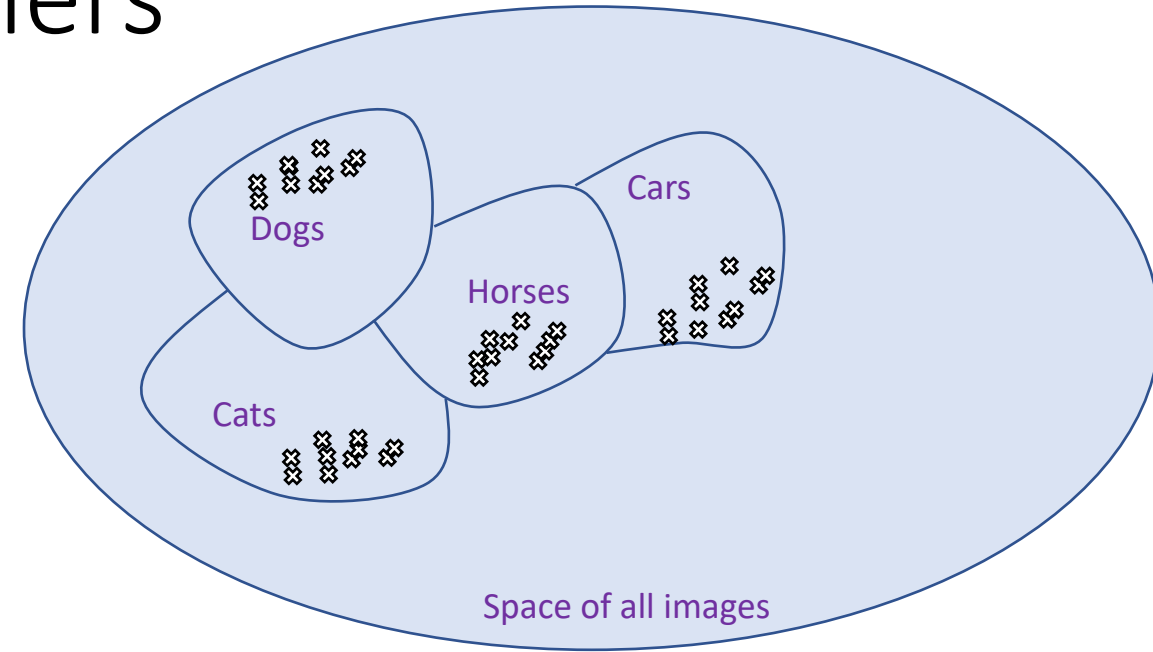


Has a clear logic, explaining the difference between dogs and cats?!
NOPE.

Impact of noise on classifiers

Misconception #1: the whole space of possible inputs was densely filled with training examples during training.

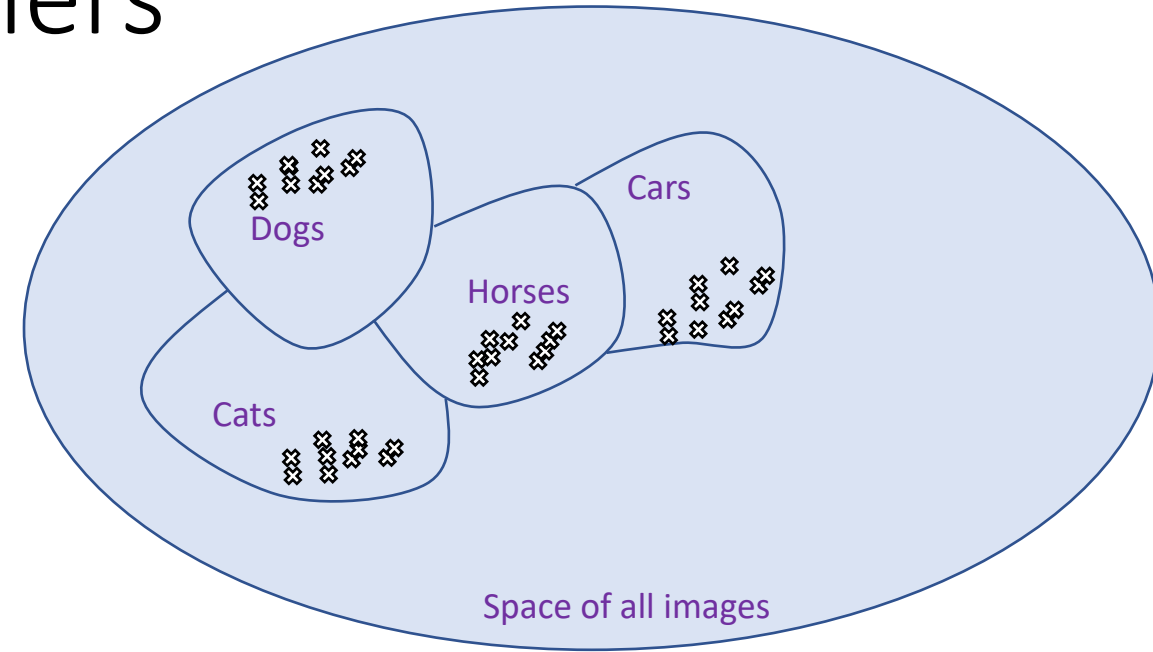
- **Correction #1:** the space is mostly noise images and sparsely filled with relevant training examples.



Impact of noise on classifiers

Misconception #1: the whole space of possible inputs was densely filled with training examples during training.

- **Correction #1:** the space is mostly noise images and sparsely filled with relevant training examples.
- Also, the training samples do not cover for all possible relevant images.

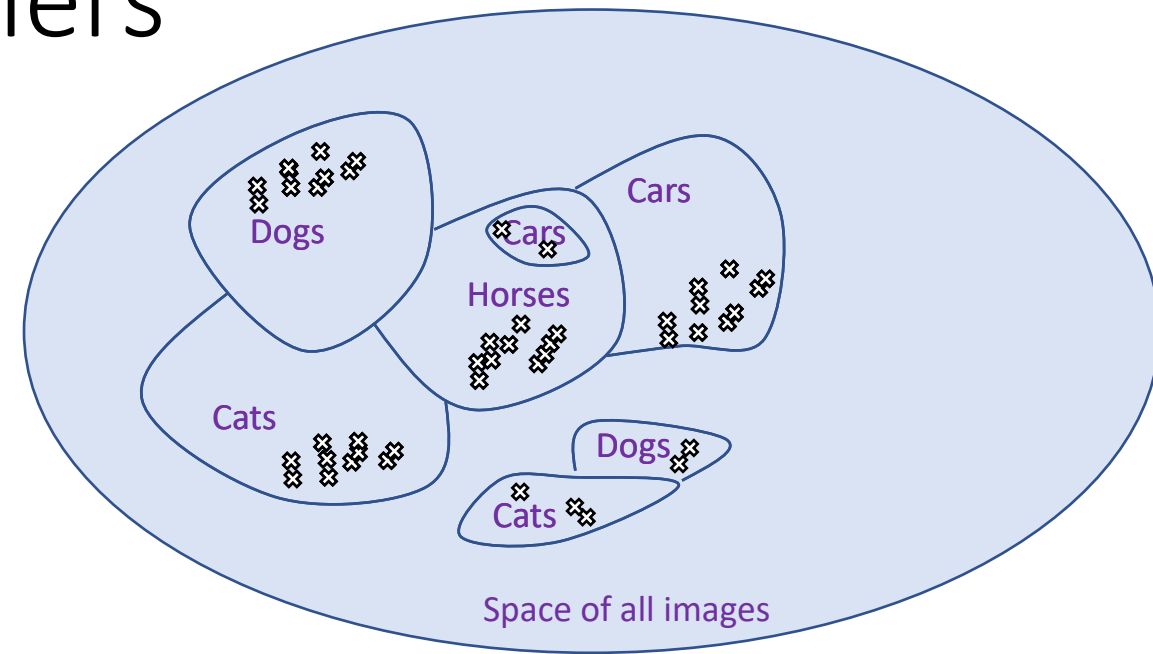


E.g. MNIST contains 28 by 28 pixels images, with pixel values in $[0, 255]$. The whole space is roughly $(256)^{28 \times 28} \approx 10^{1888}$ images, and most of them are noise. The MNIST dataset contains 60000 images only.

Impact of noise on classifiers

Misconception #2: regions are contiguous and filled with samples.

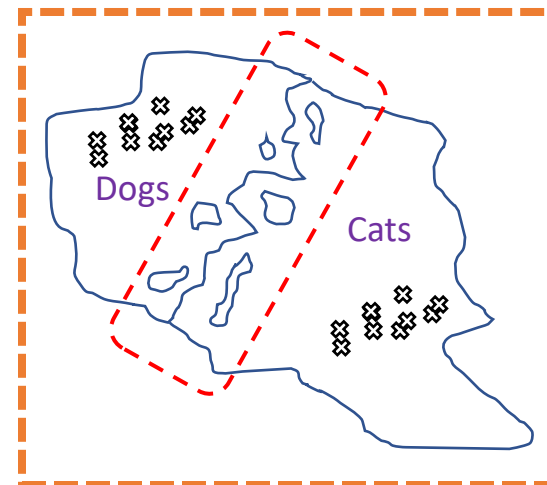
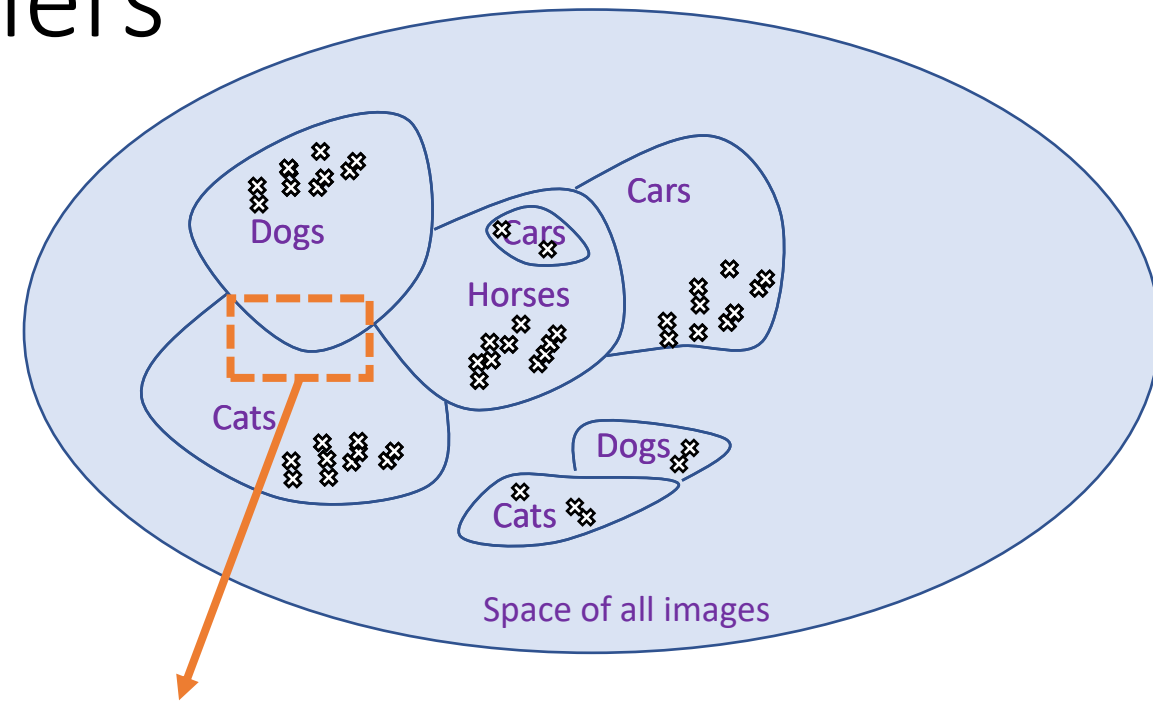
- **Correction #2:** regions will not necessarily be contiguous.



Impact of noise on classifiers

Misconception #2: regions are contiguous and filled with samples.

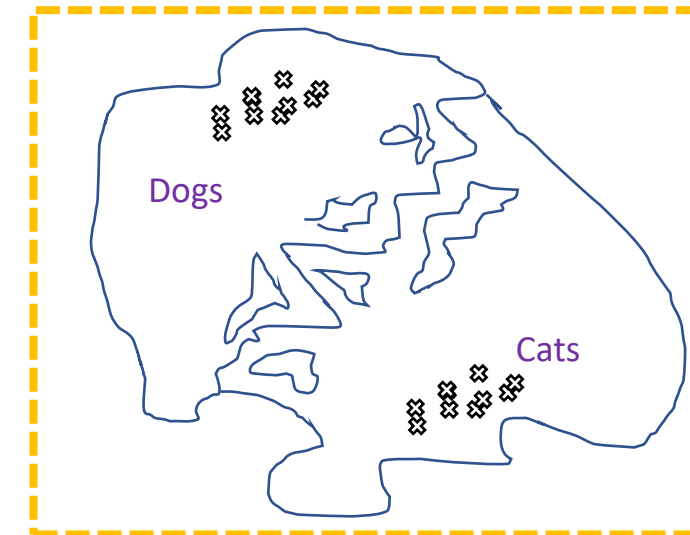
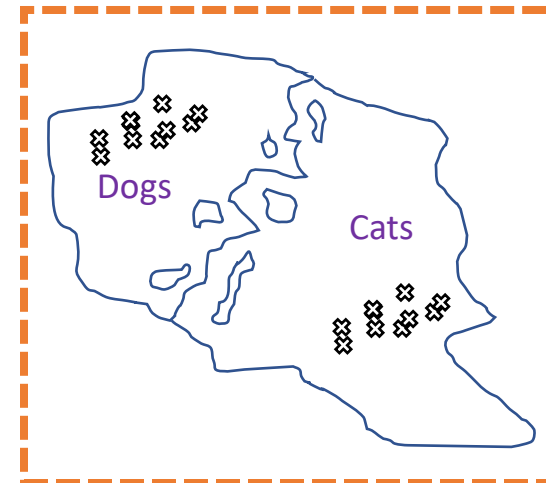
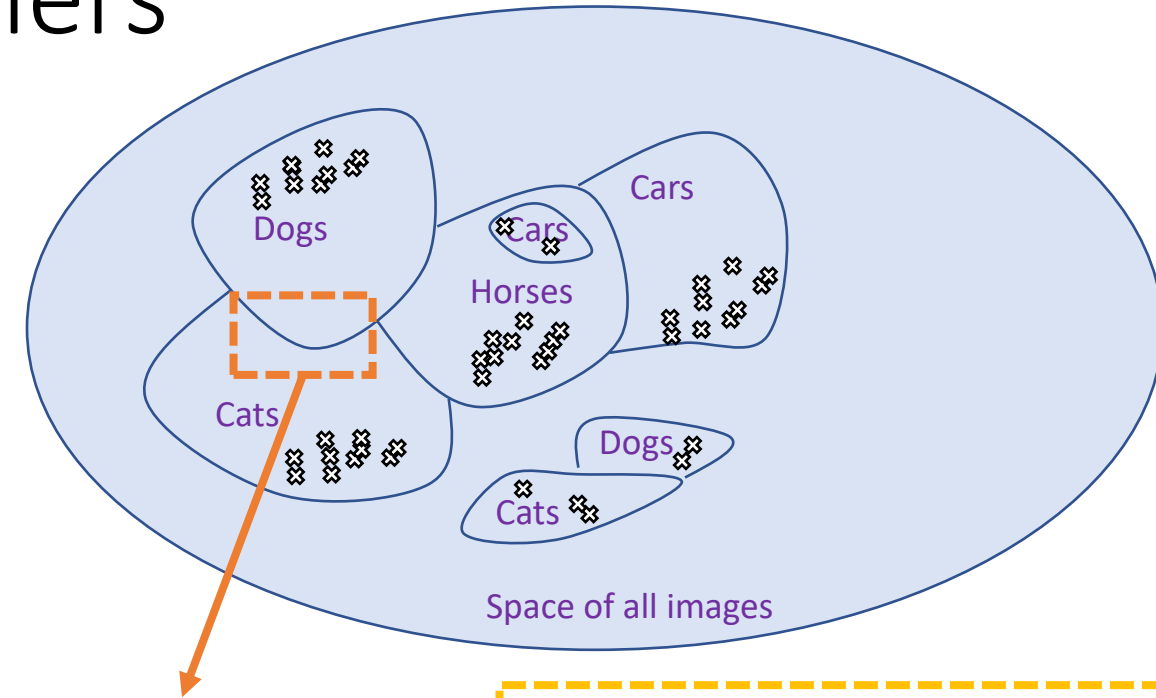
- **Correction #2:** regions will not necessarily be contiguous. Boundaries between classes might even be very **erratic**!



Impact of noise on classifiers

Misconception #3: the decision boundaries between classes are smooth and make perfect sense.

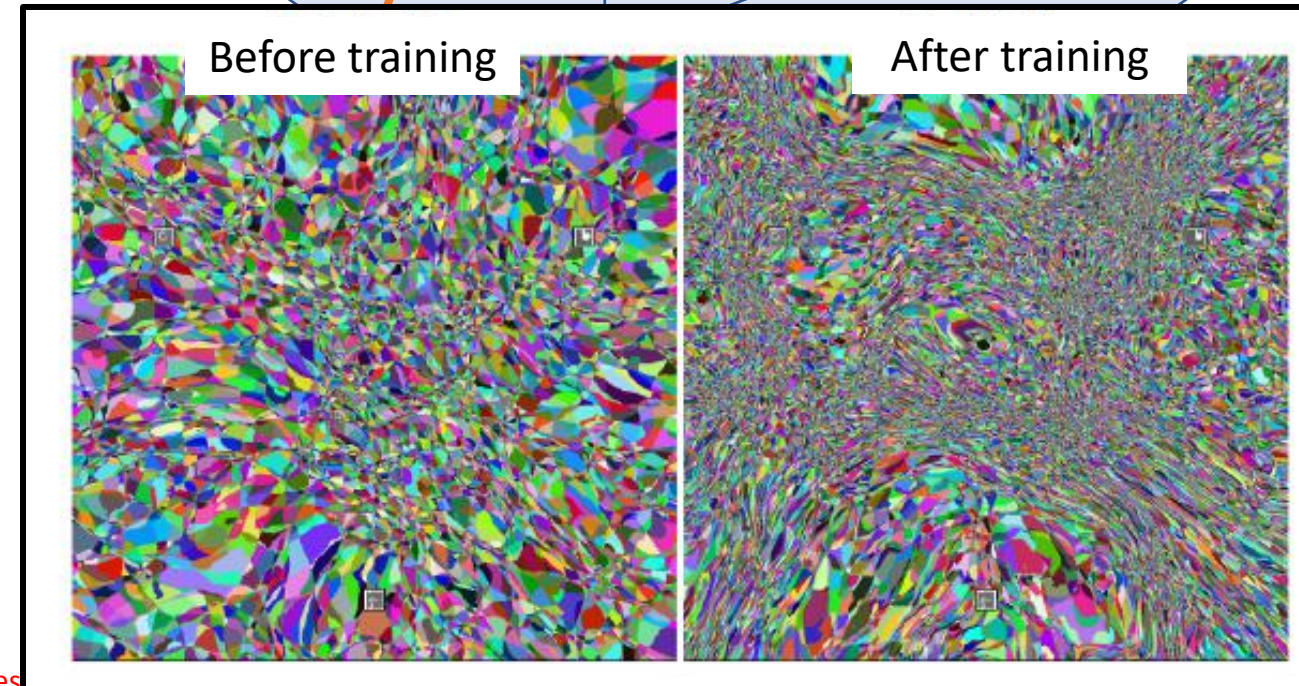
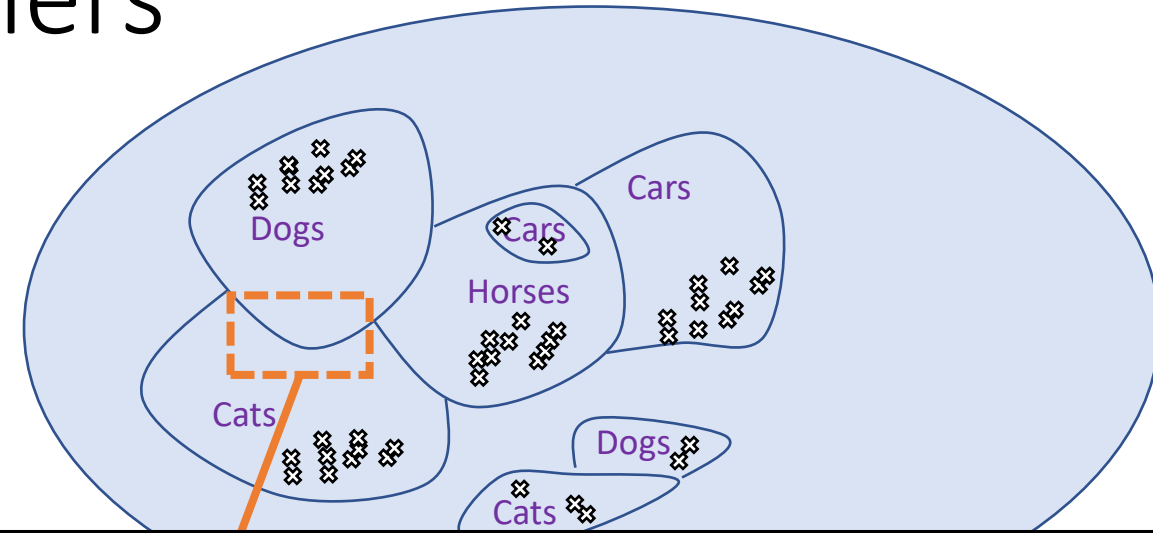
- **Correction #3:** In fact, the boundaries between samples are often “randomly” decided.
- On different epochs, the boundaries might change somewhat randomly (?!).



Impact of noise on classifiers

Misconception #3: the decision boundaries between classes are smooth and make perfect sense.

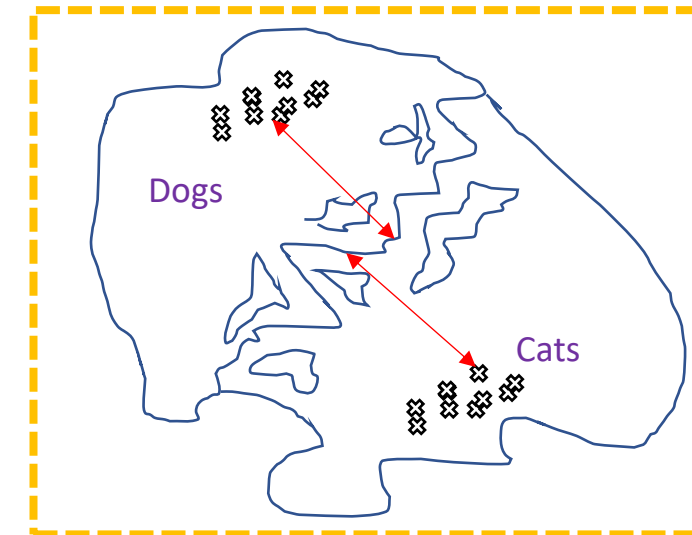
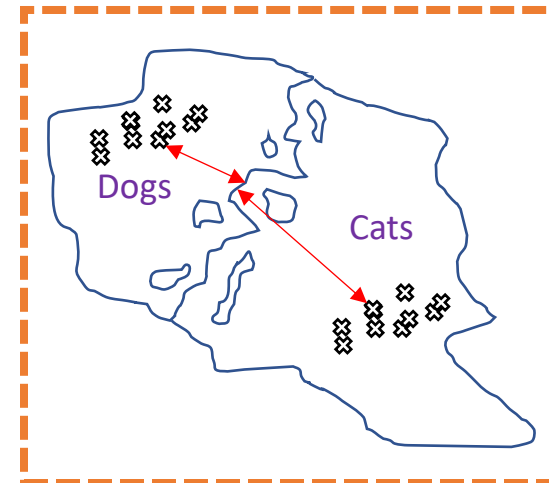
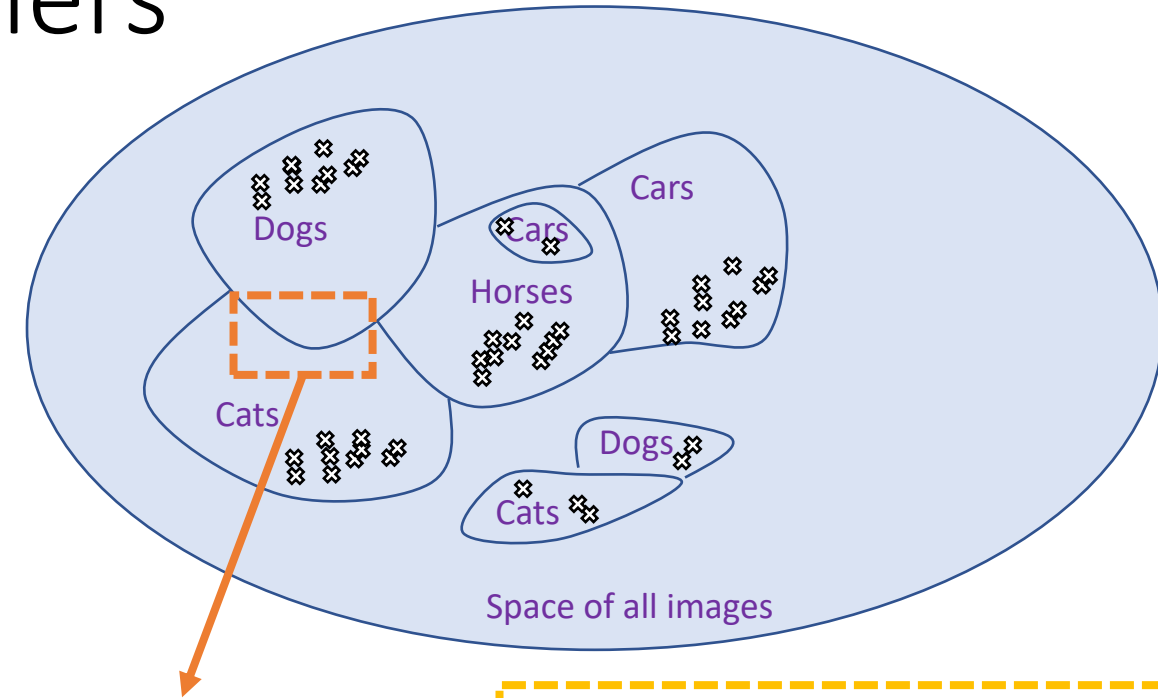
- **Correction #3:** In fact, the boundaries between samples are often “randomly” decided.
- On different epochs, the boundaries might change somewhat randomly (?!).



Impact of noise on classifiers

Misconception #3: the decision boundaries between classes are smooth and make perfect sense.

- **Correction #3-bis:** Training samples are often condensed, far away from the boundaries.



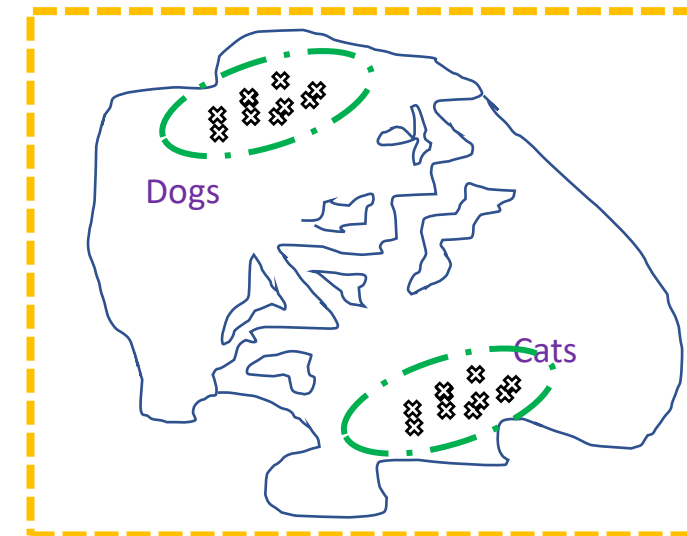
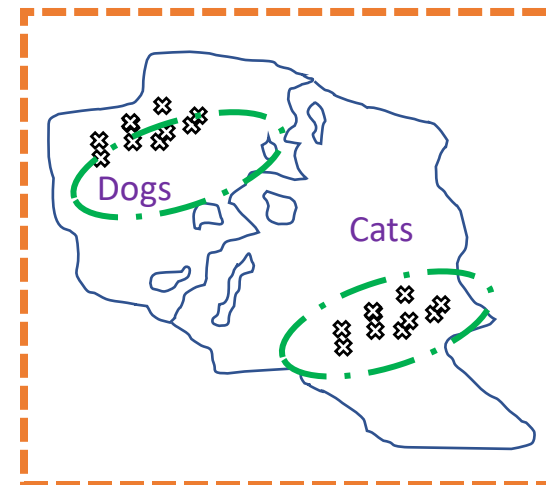
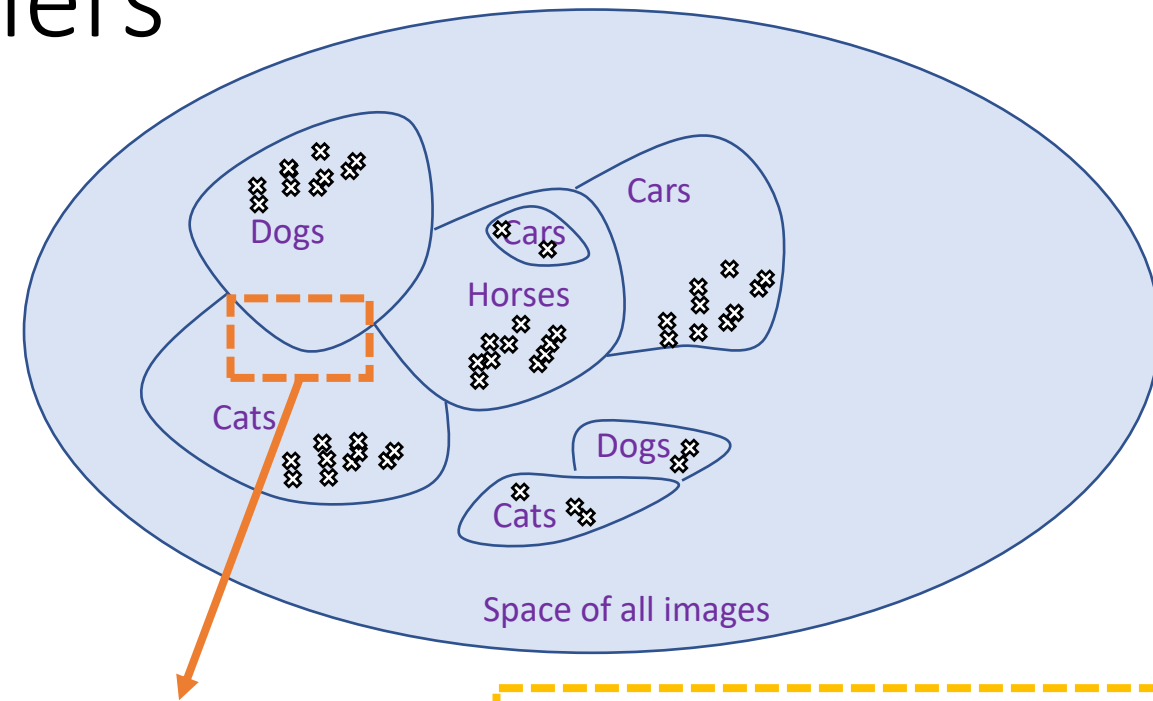
Impact of noise on classifiers

Correction #3-bis: Training samples are often condensed, far away from the boundaries.

Definition (manifolds):

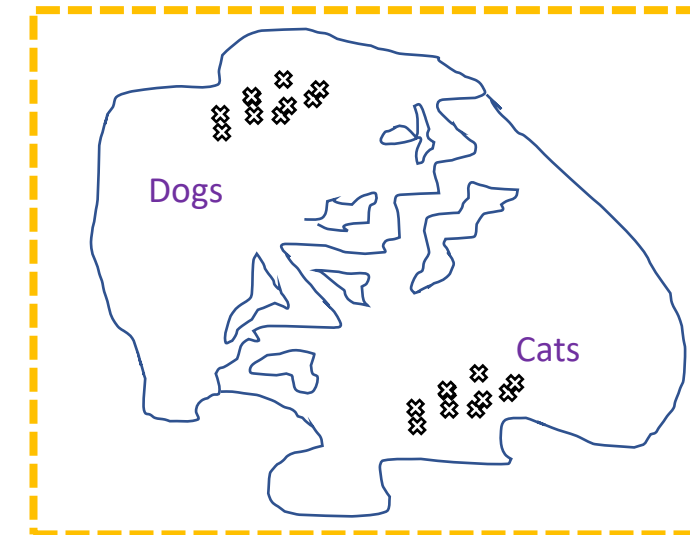
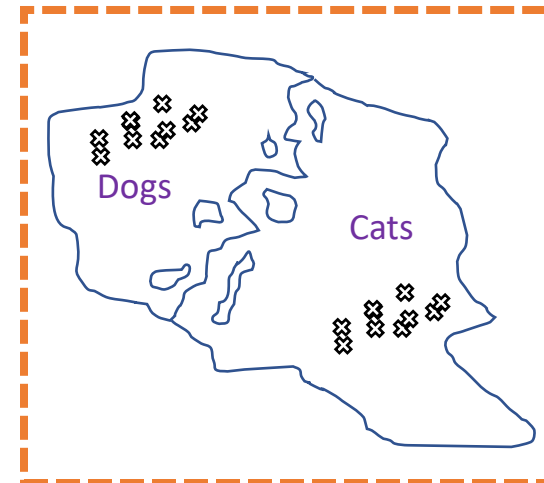
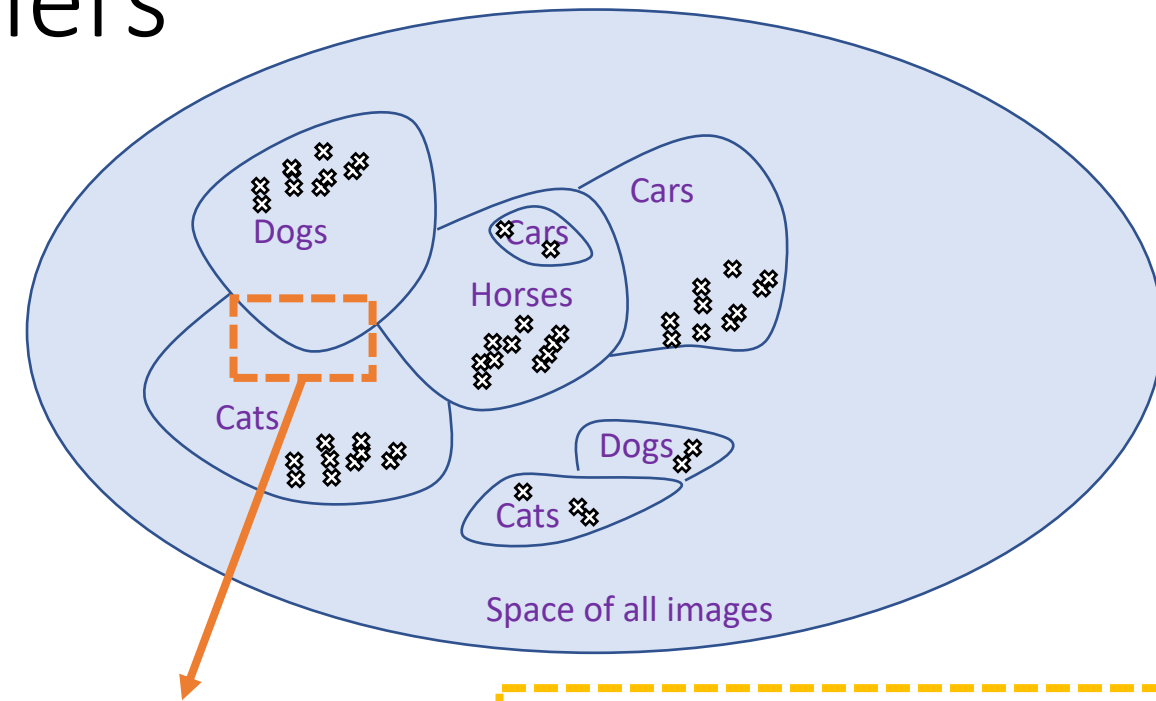
These small regions containing a large number of training examples are mathematically called **manifolds**.

See [TDS1], if curious.



Impact of noise on classifiers

- **Misconception #3:** the decision boundaries between classes are smooth and make perfect sense.
- **Correction #3-bis:** Training samples are often condensed, far away from the boundaries.
- Boundaries decided by the Deep Learning models often exhibit the same behavior as the **Support Vector Machines** boundaries. But in a more random manner.

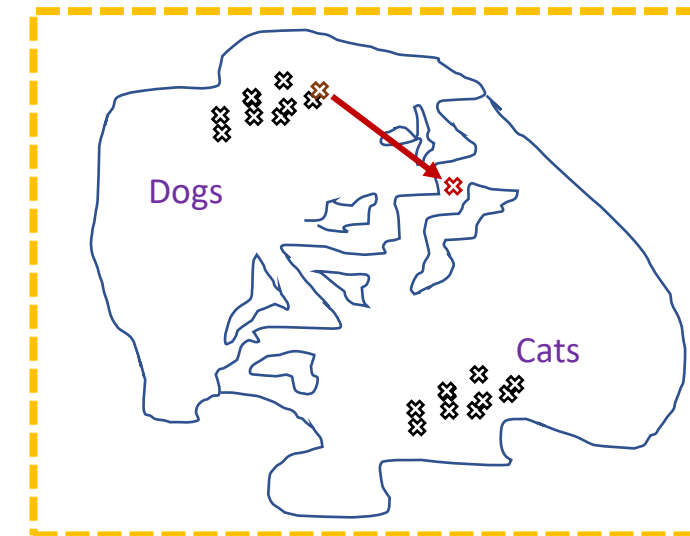
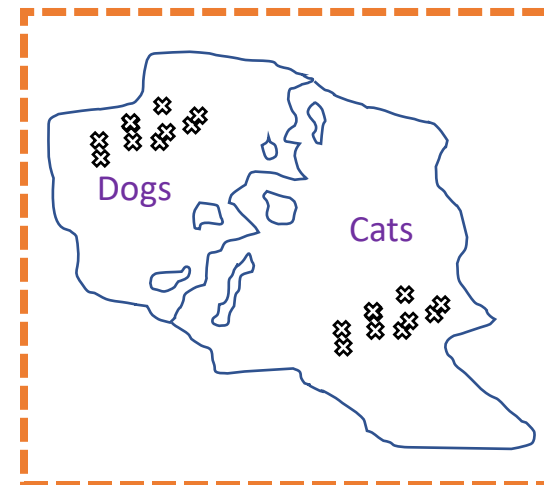
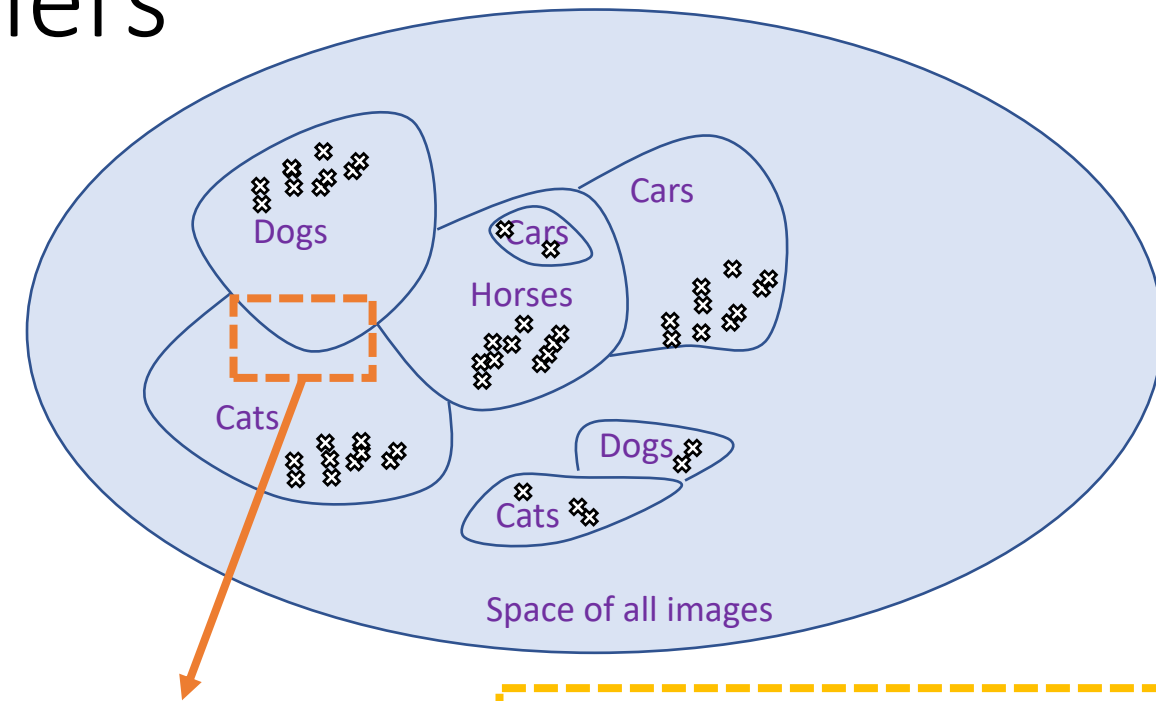


Impact of noise on classifiers

ENM procedure, explained:

When randomly noising an **original sample** to make an **attack sample**, we move randomly in the feature map.

- We may even move in the boundary region, where the sample might become misclassified.
- The **attack sample** will therefore look similar to a dog picture, but will be misclassified as a cat.



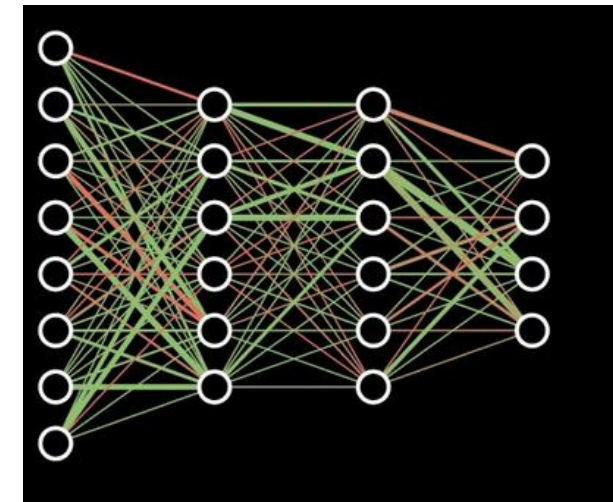
Lessons from ENM attacks

Here is my intuition about attacks.
(Not claiming it is the universal truth.)

- In Deep Learning, to **learn** means to **understand that some objects are similar on certain aspects/features** and **use those similarities for classification**.
- It means to discard irrelevant information which does not contribute to understand those similarities.



Cat or Dog Picture (high dimensional input)



Neural Network, processing input image



Is the **presence of fur** relevant for discerning cats from dogs? **No, discard info.**



Is the **shape of eyes** relevant for discerning cats from dogs? **Yes, process info.**

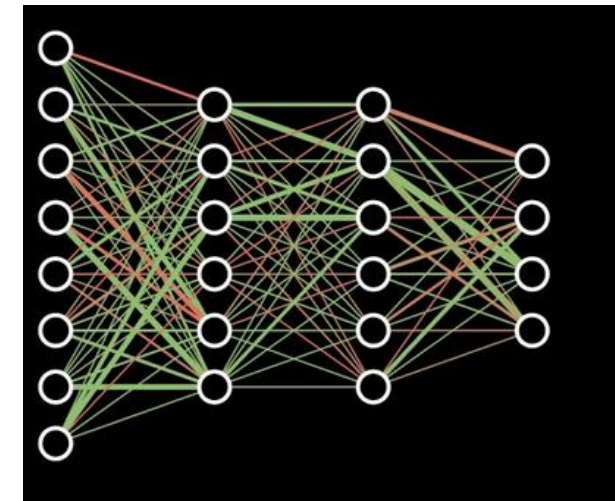
Lessons from ENM attacks

Discarding information is achieved by progressively **compressing input data into a space of lower dimensionality** using several Neural Networks layers.

- A linear neuron would achieve this, by mapping inputs onto n output with less features, i.e. a lower-dimension hyperplane.
- During training, it learns to perform useful projections of the training data.



Cat or Dog Picture (high dimensional input)



Neural Network, processing input image



$$\mathbf{a} = [a_1 \quad a_2 \quad \dots \quad a_n]$$

Lower dimensional vector produced by final hidden layer of Neural Network.

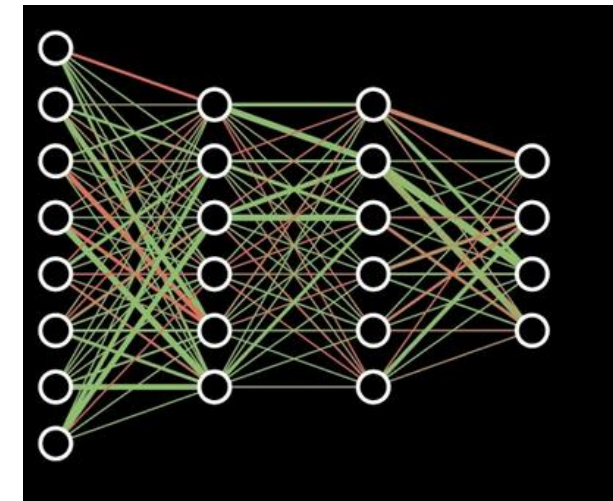
Lessons from ENM attacks

Neural Networks then aim to map similar objects close together in this low dimensionality space.

- For instance, all pictures of cats will produce projection vectors a , with roughly similar values. Same thing for dogs pictures.
- Cats vectors and dogs vectors will, however, be very different!
- Final layer then implements a binary decision on vector a .



Cat or Dog Picture (high dimensional input)



Neural Network, processing input image



$$\mathbf{a} = [a_1 \quad a_2 \quad \dots \quad a_n].$$

Lower dimensional vector produced by final hidden layer of Neural Network.

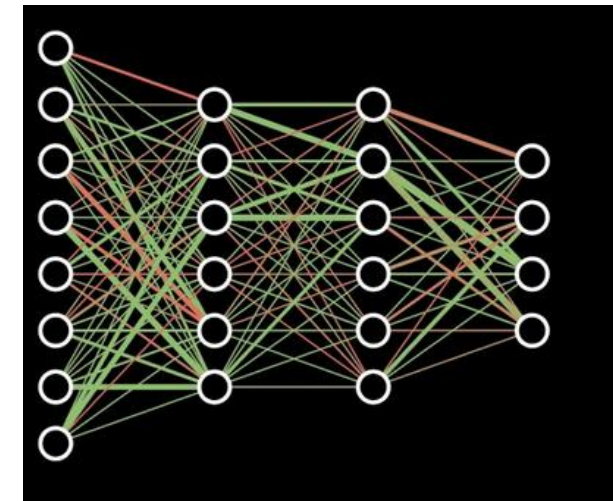
Lessons from ENM attacks

- Due to compression, there are many directions in a high dimension feature space along which a small step might lead to big changes in predictions.
- In zones with low training data densities, the decision boundaries can lie very close together, because they were never properly learned from training samples!



Cat or Dog Picture (high dimensional input)

Small
change
here...



Neural Network, processing input image

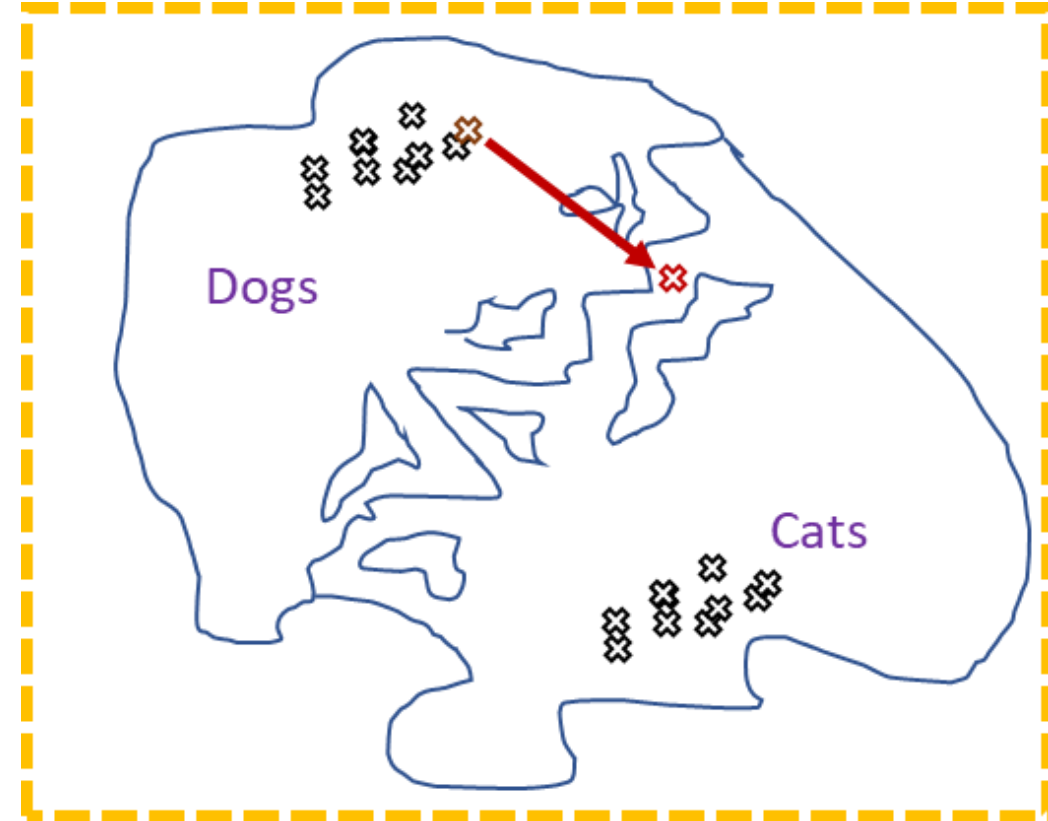
... Could
result in a
huge
change
here!

$$\mathbf{a} = [a_1 \quad a_2 \quad \dots \quad a_n]$$

Lower dimensional vector produced by final hidden layer of Neural Network.

Lessons from ENM attacks

- In zones with low training data densities, some small changes on an input can then lead to big changes in predictions produced by the trained Neural Network.
- And that is what we exploit to generate **attack samples**!



“panda”
57.7% confidence

+ .007 ×



“nematode”
8.2% confidence

=



“gibbon”
99.3 % confidence

Lessons from ENM attacks

- **Unfortunately, this means that all deep learning models will always be susceptible to attacks and small changes on inputs data.**
(And that is something we have to accept.)

Lessons from ENM attacks

→ Unfortunately, this means that all deep learning models will always be susceptible to attacks and small changes on inputs data.

(And that is something we have to accept.)

So what? Is that it then?

Are Neural Networks flawed beyond repair?

Are we giving up on Neural Networks then?

Reason #2: Defense

Definition (**Defense** on Neural Networks):

In adversarial machine learning, **defense** refers to machine learning techniques that attempt to **protect models from being attacked** by malicious attempts.

Important: defense mechanisms often rely on an understanding of how attacks work.

SOMETHING
FOR LATER...

Let us call it a break for now

We will continue on the next lecture with more on Adversarial ML

Learn more about these topics

Out of class, supporting papers, for those of you who are curious.

- [Xie2017] Xie et al., “Adversarial Examples for Semantic Segmentation and Object Detection”, 2017.
<https://arxiv.org/abs/1703.08603>
- [Szegedy2013] **Szegedy** et al., “Intriguing properties of neural networks”, 2013.
<https://arxiv.org/abs/1312.6199>

Learn more about these topics

- [Moosavi2017] **Moosavi-Dezfooli** et al., “Universal adversarial perturbations”, 2017.
<https://arxiv.org/abs/1610.08401>
- [Hayes2017] Hayes et al., “Learning Universal Adversarial Perturbations with Generative Models”, 2017.
<https://arxiv.org/abs/1708.05207>
- [Goodfellow2018] **Goodfellow** et al., “Making machine learning robust against adversarial inputs”, 2018.
<https://dl.acm.org/doi/10.1145/3134599>

Learn more about these topics

Some extra (easy) reading and videos for those of you who are curious.

- [Verge1] Google's AI thinks this turtle looks like a gun, which is a problem:

<https://www.theverge.com/2017/11/2/16597276/google-ai-image-attacks-adversarial-turtle-rifle-3d-printed>

- [Spectrum1] Slight Street Sign Modifications Can Completely Fool Machine Learning Algorithms:

<https://spectrum.ieee.org/cars-that-think/transportation/sensors/slight-street-sign-modifications-can-fool-machine-learning-algorithms>

Learn more about these topics

- [Verge2] These glasses trick facial recognition software into thinking you're someone else:
<https://www.theverge.com/2016/11/3/13507542/facial-recognition-glasses-trick-impersonate-fool>
- [YTB1] Defeating Facial Recognition:
<https://www.youtube.com/watch?v=tbdcl5Ux-9Y>
- [TDS1] Manifolds in Data Science — A Brief Overview:
<https://towardsdatascience.com/manifolds-in-data-science-a-brief-overview-2e9dde9437e5>